

RetryGuard on TopFull — Mentor Update

What this is: a progress update for our mentors, not the final report. It covers the infrastructure we ran on, the test application (Online Boutique), the scenarios we ran, and the dry results gathered so far from our 48-run paper-grade campaign. Each results section shows charts with short factual observations — no conclusions or recommendations yet. Use this to tell us what looks sufficient and what's missing.

Data source: `experiments/results/campaign_48/` (48 runs, all collectors enabled, 3 repeats per condition).

1. Infrastructure & Environment

This cluster is based on **Ron Nezer's** working TopFull environment: he provided three VMs already running TopFull, and we copied them into this project's GCP account, then made the changes needed for this workshop (Istio retry control, RetryGuard, collectors, experiment runner).

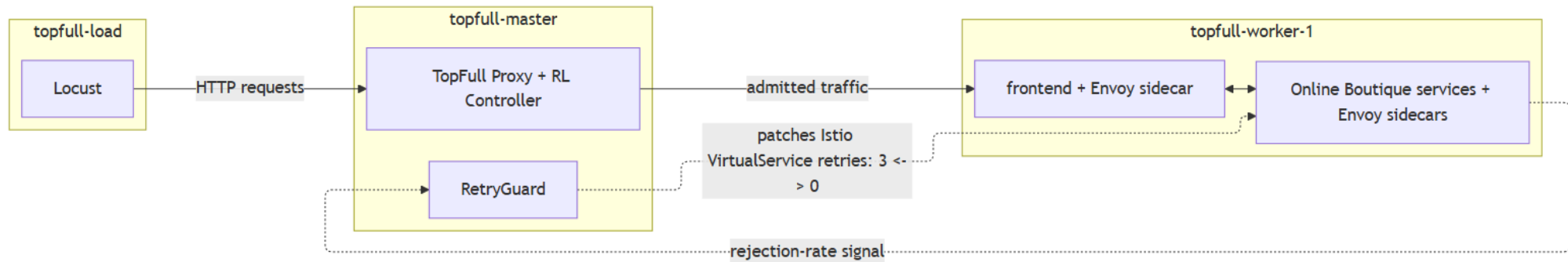
That inherited stack runs every Online Boutique service at a fixed **1 replica** (TopFull's `instance_scaling.py`, on a single worker node rather than the paper's five). We therefore did not take autoscaling or replica-count changes into account: there is nothing to scale down from, over-scaling from retries cannot show up, and Scenarios 3/4 create bottlenecks with a CPU limit instead of reducing replicas.

We run on those 3 Google Cloud VMs (project `networks-workshop`, zone `us-central1-a`):

VM	Role
<code>topfull-master</code>	Kubernetes control plane, Istio control plane, TopFull's proxy + RL rate controller, RetryGuard
<code>topfull-worker-1</code>	All Online Boutique pods, each with an Envoy sidecar (Istio's service mesh proxy)
<code>topfull-load</code>	Locust load generator

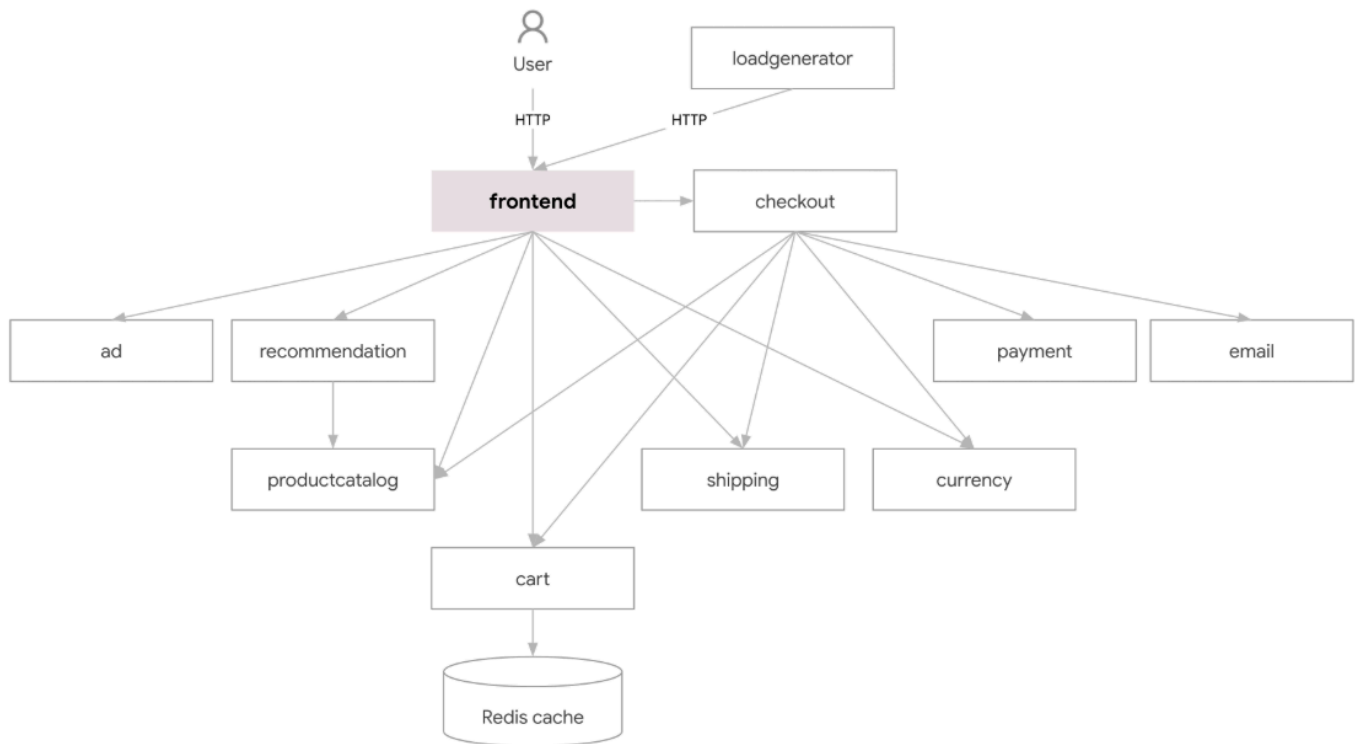
The stack is a self-managed Kubernetes cluster (`kubeadm`, not a managed GKE cluster) running Istio as the service mesh. Every Online Boutique service pod has an Envoy sidecar that Istio injects; all inter-service traffic goes through these sidecars, which is what makes Istio's retry policy (and RetryGuard's control over it) possible.

- **TopFull** sits in front of the cluster as an entry-point proxy plus a reinforcement-learning controller that throttles admitted traffic per API when it detects overload.
- **RetryGuard** (our own implementation, built from its paper) watches per-service rejection rates via the Envoy sidecars and disables/re-enables Istio's retry policy (`attempts: 3` → `attempts: 0` and back) on a per-service basis when rejection crosses a threshold.
- **Locust** generates the offered load from `topfull-load`, simulating users browsing/buying on Online Boutique.



2. Online Boutique — Role & Architecture

[Online Boutique](#) is Google's reference e-commerce microservices demo — a real, polyglot, 10+ service application with a realistic call graph, which is why it's a good stand-in for "a production-like microservice system" in this workshop rather than a toy app.



Services we specifically constrain (CPU-limit) in later scenarios to create a controlled bottleneck:

Service	Constrained in	Position in the call graph
checkoutservice	Scenario 3	Critical path: Frontend → Checkout →
productcatalogservice	Scenario 4A	Gateway-adjacent — Frontend calls it directly on many product-browse paths
paymentservice	Scenario 4B	Indirect — reachable only via Frontend → Checkout → Payment

All services stay at that fixed 1 replica in both conditions, so any effect we see is due to retry behavior and CPU limits, not replica count changes.

3. Scenarios — High Level

Each scenario is run under two conditions with identical load: **Baseline** (TopFull only, Istio default retries, RetryGuard off) and **RetryGuard** (same, plus RetryGuard toggling retries per service). Every combination is repeated 3 times (Locust traffic is non-deterministic).

#	Scenario	What changes	Duration	Open question(s) targeted
1	Normal Operation	Flat load, well within capacity	5 min	Sanity check — RetryGuard should stay inert
2	Sustained Overload	Peak load from t=0, held flat	10 min	System-level gains, topology beneficiaries, chain propagation
3	Targeted Bottleneck	<code>checkoutservice</code> CPU-limited	10 min	Topology beneficiaries, chain propagation
4A / 4B	Topology Position	<code>productcatalogservice</code> (A) vs <code>paymentservice</code> (B) CPU-limited	10 min	Topology position sensitivity
5	Re-enable Interval Tuning	Same load as S6; re-enable window swept 10/20/30/60s	15 min	Interval parameter sensitivity
6	Forced Recovery	Peak 5 min, then ~25% load for 10 min	15 min	Combined equilibrium; also the reference for Scenario 5

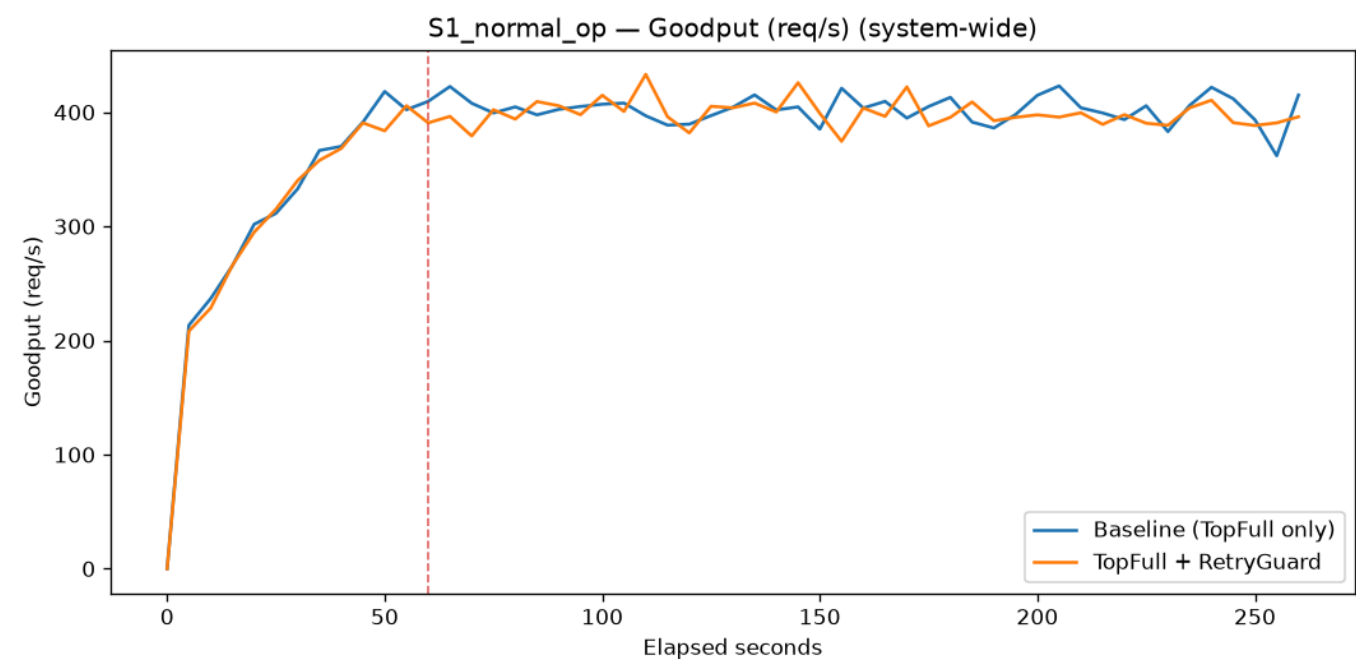
Section 4 walks through the dry results for each of these, grouping Scenario 5 into Scenario 6's subsection since S5 has no baseline of its own — it's only meaningful compared against S6.

4. Results — Per-Scenario Deep Dive

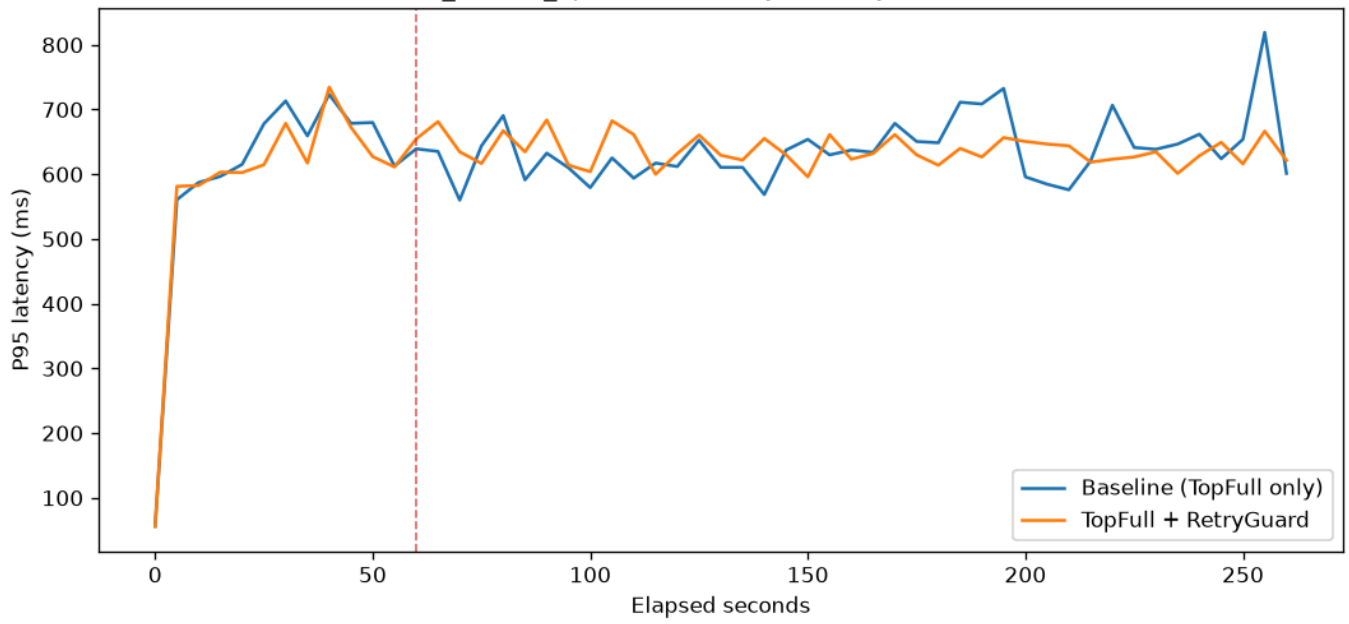
All charts below plot **Baseline** (mean of 3 repeats) against **RetryGuard** (mean of 3 repeats), x-axis in elapsed seconds. Where RetryGuard fired, vertical dashed lines mark disable (`ON→OFF` , red) and re-enable (`OFF→ON` , green) events. Additional per-endpoint/per-service charts not shown here are in `charts_gallery/<scenario>/` .

4.1 Scenario 1 — Normal Operation

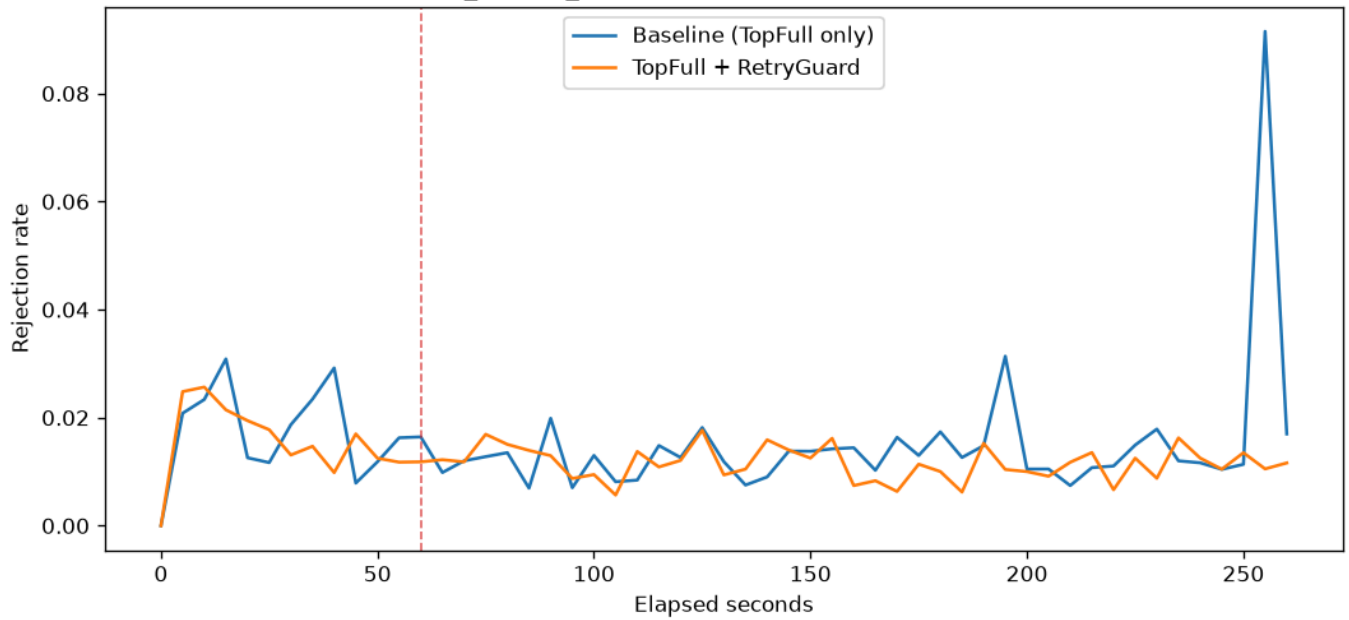
Sanity check: under light load, both conditions should behave (almost) identically and RetryGuard should make no or minimal changes.



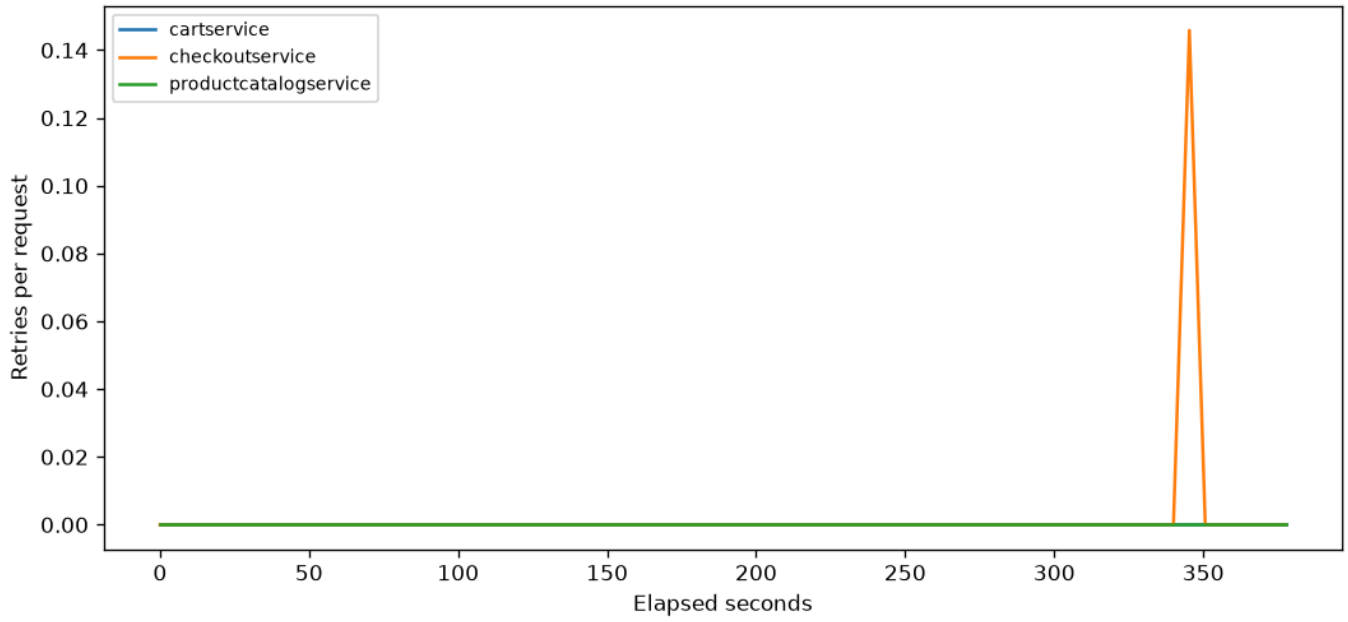
S1_normal_op — P95 latency (ms) (system-wide)



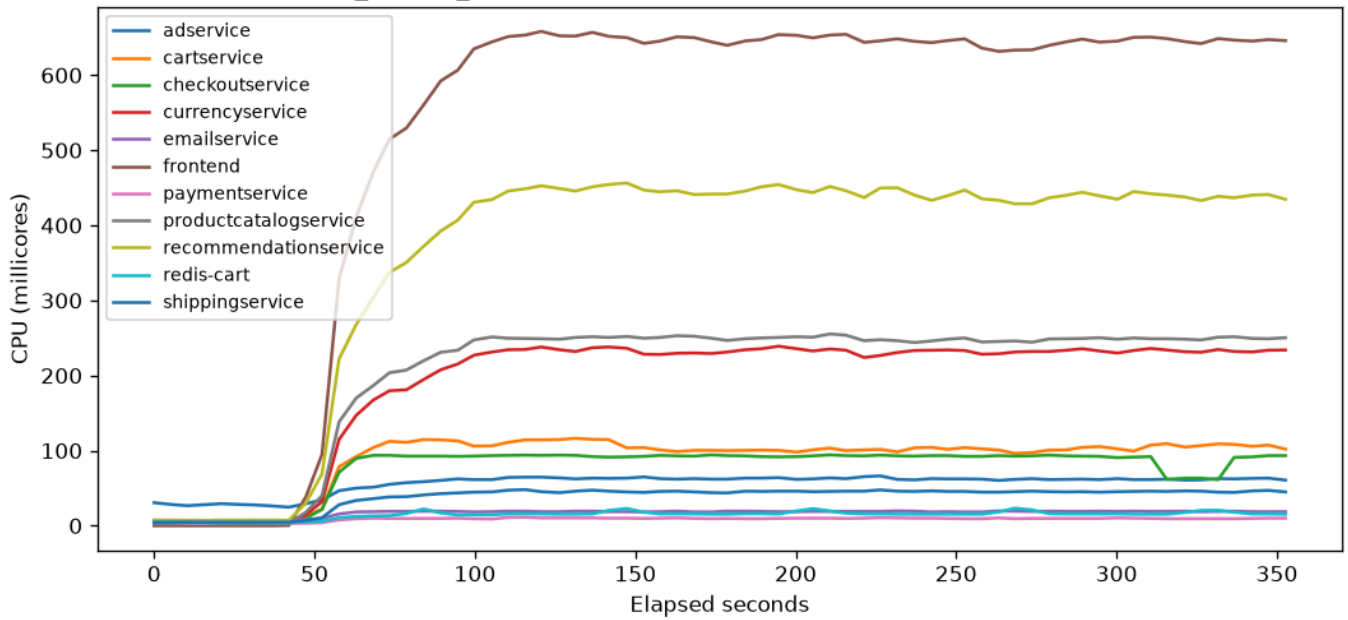
S1_normal_op — Rejection rate (system-wide)

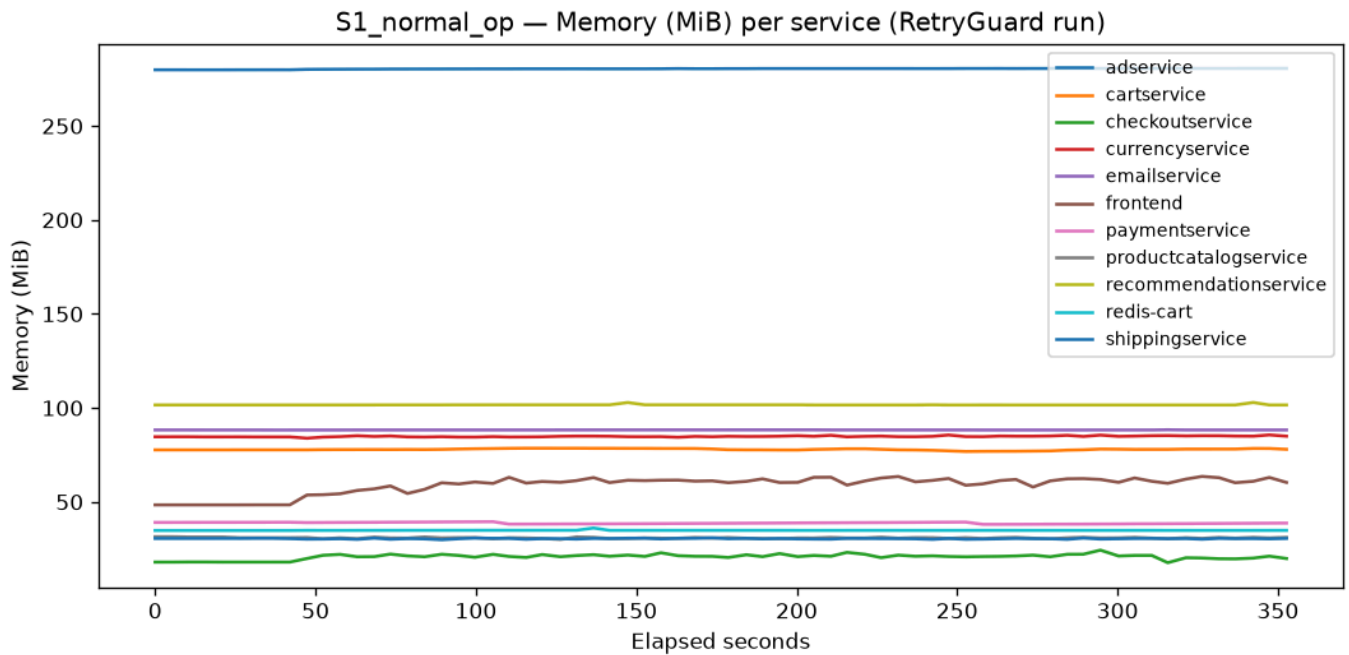


S1_normal_op — envoy_retries_frontend retries/request per target



S1_normal_op — CPU (millicores) per service (RetryGuard run)





What we observe:

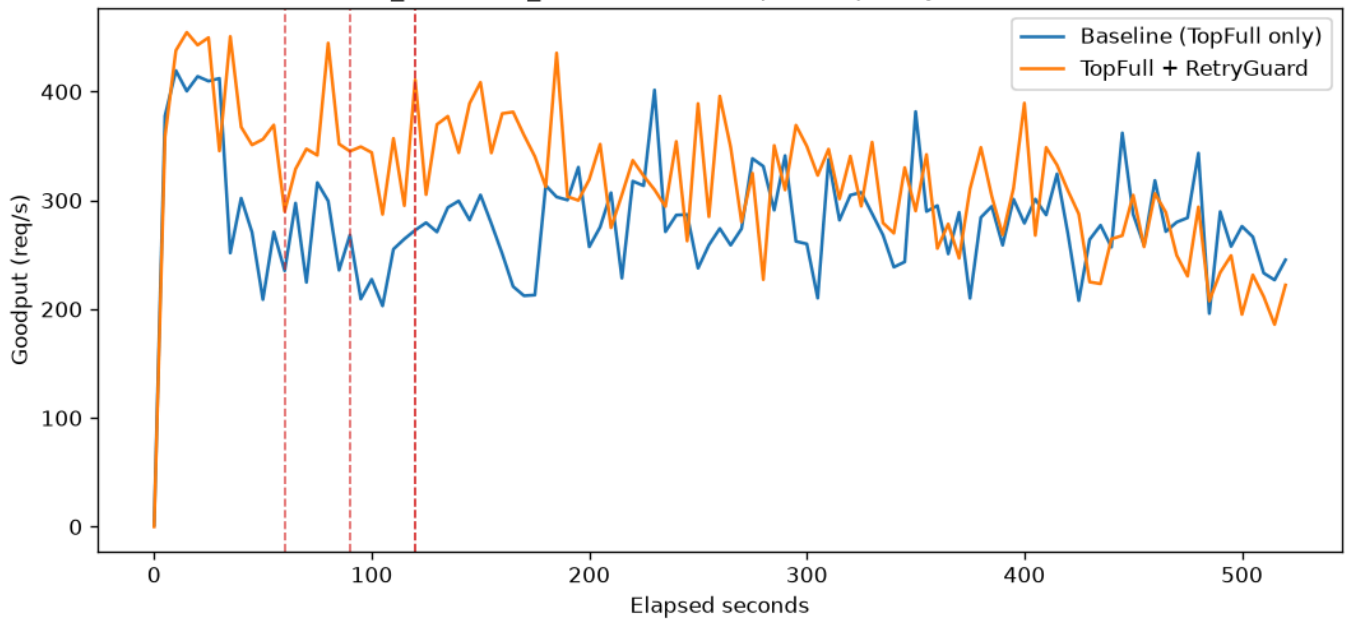
- Baseline and RetryGuard system-wide goodput overlap around ~400 req/s after a shared ramp in the first ~50s, and neither line steps at the 60s marker.
- Rejection is not identically near zero in both conditions. RetryGuard stays in a narrow ~0.01–0.02 range, while baseline is mostly low but shows several spikes (mean ~0.04–0.09).
- A single red `ON→OFF` marker sits at ~60s on the S1 charts; there is no green `OFF→ON`. That matches the campaign logs: one `checkoutservice ON→OFF` on each RetryGuard repeat and no re-enable.

Additional endpoint-level charts: [charts_gallery/S1_normal_op/](#).

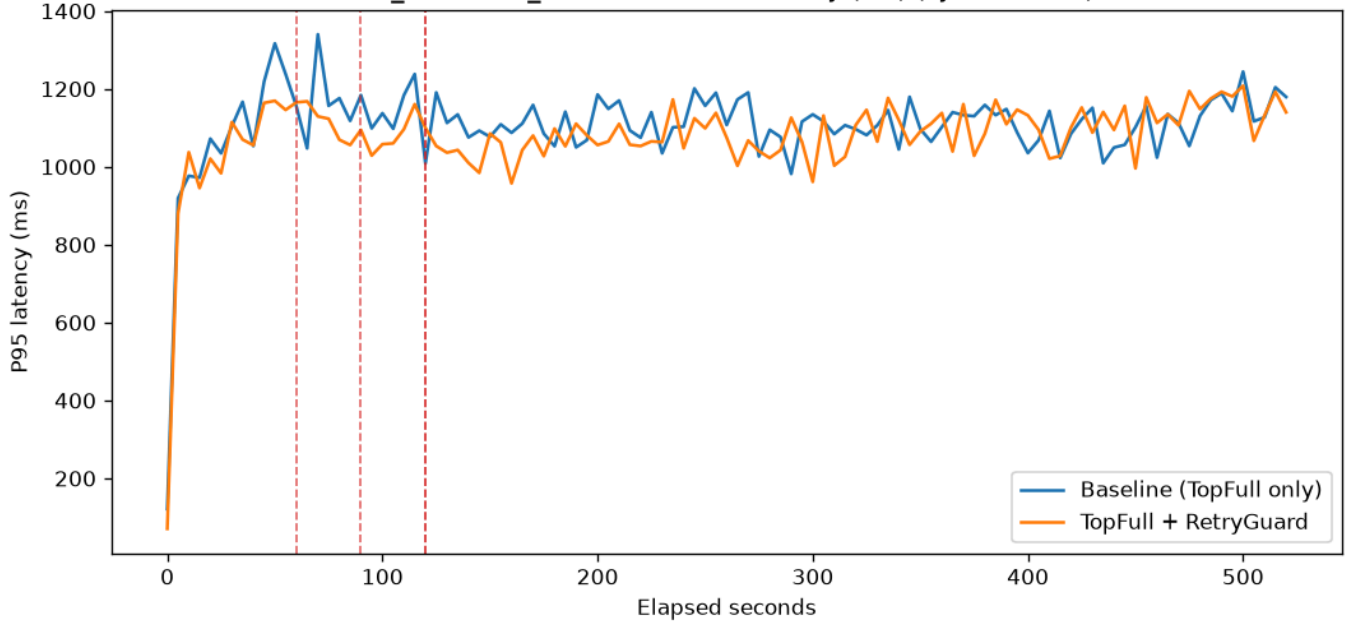
4.2 Scenario 2 — Sustained Overload

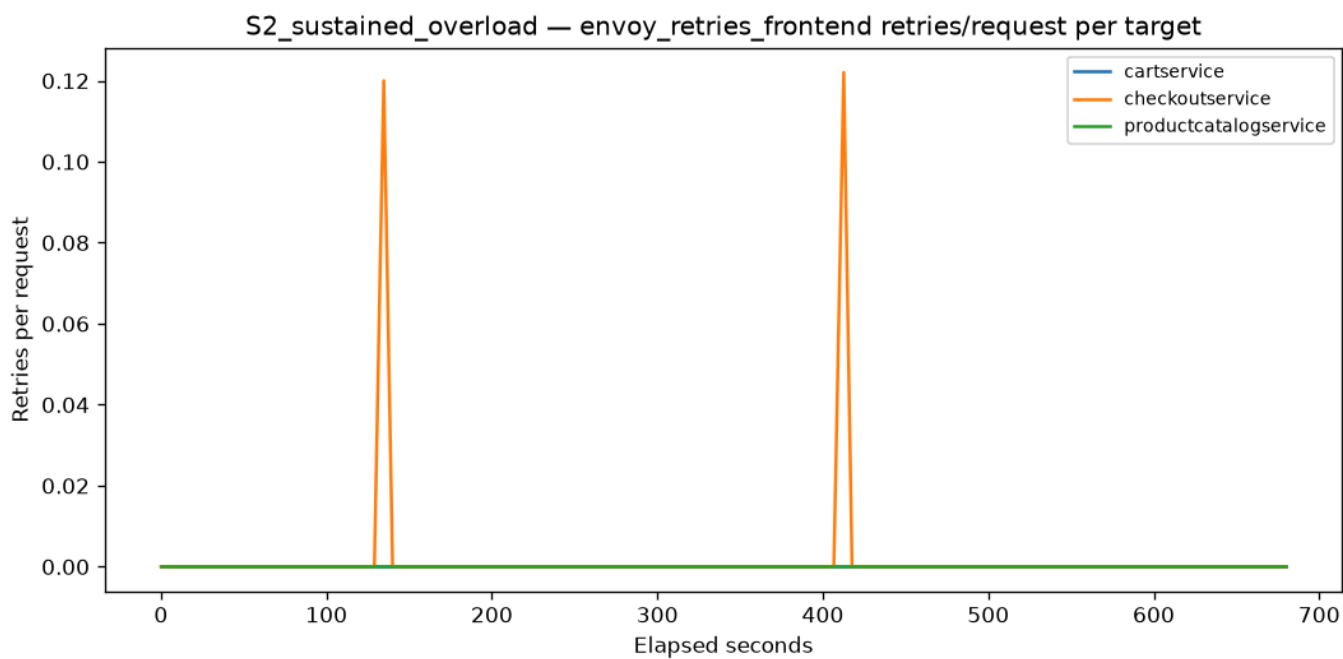
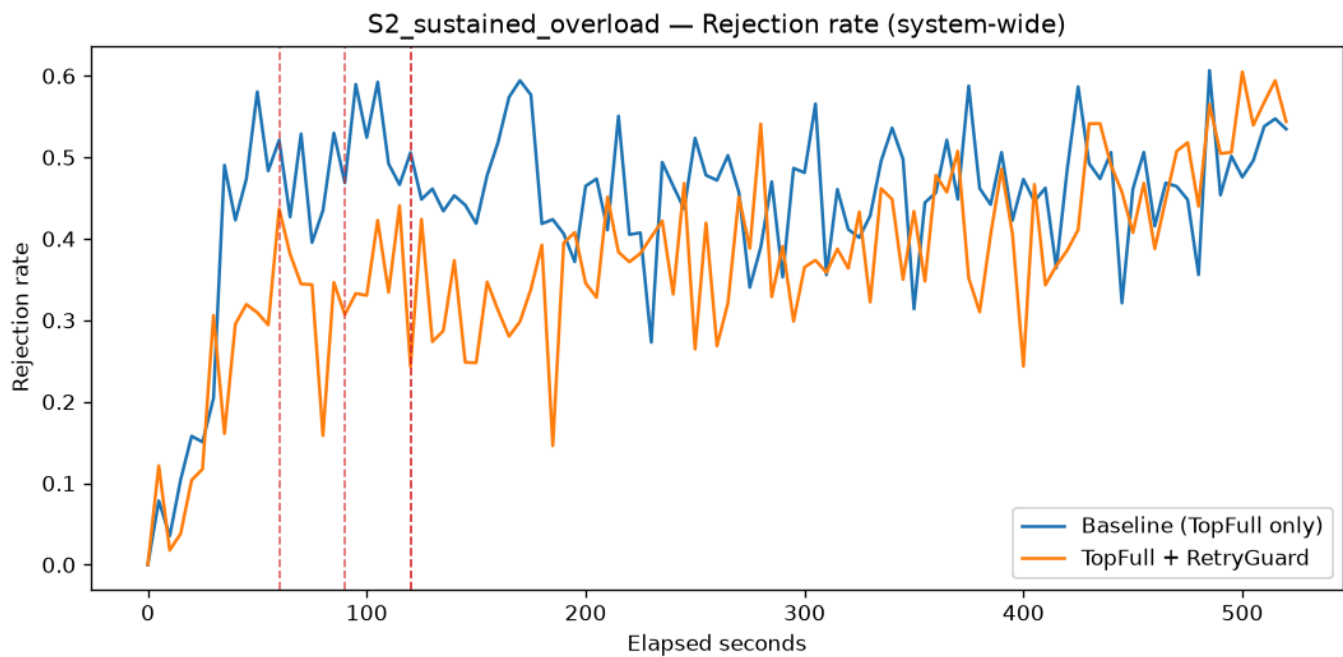
Peak load held flat for 10 minutes — the core "does suppressing retries help under sustained overload" test.

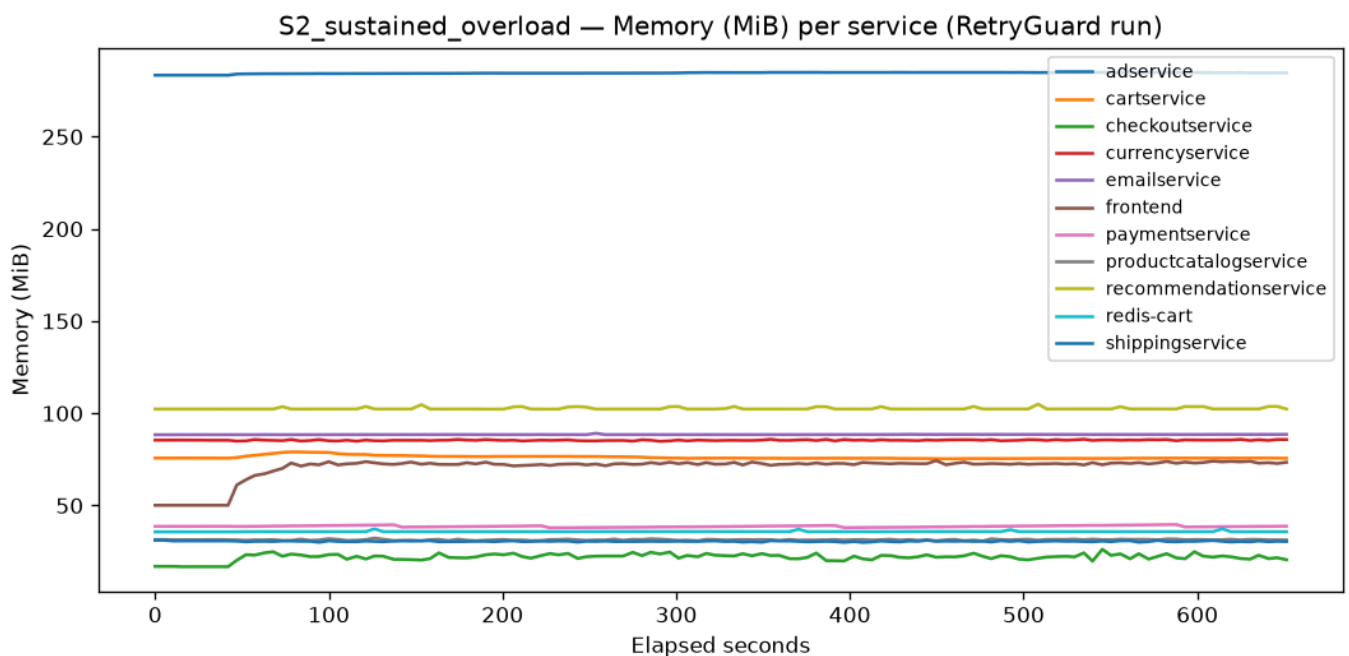
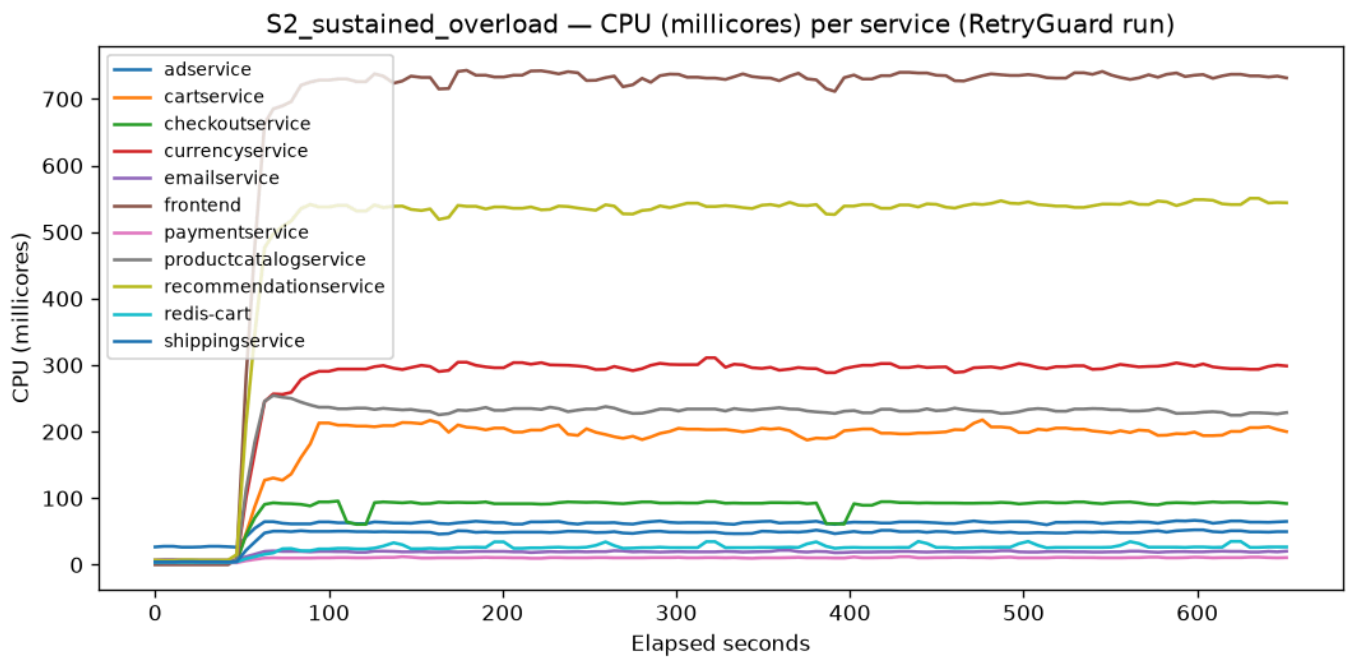
S2_sustained_overload — Goodput (req/s) (system-wide)



S2_sustained_overload — P95 latency (ms) (system-wide)







What we observe:

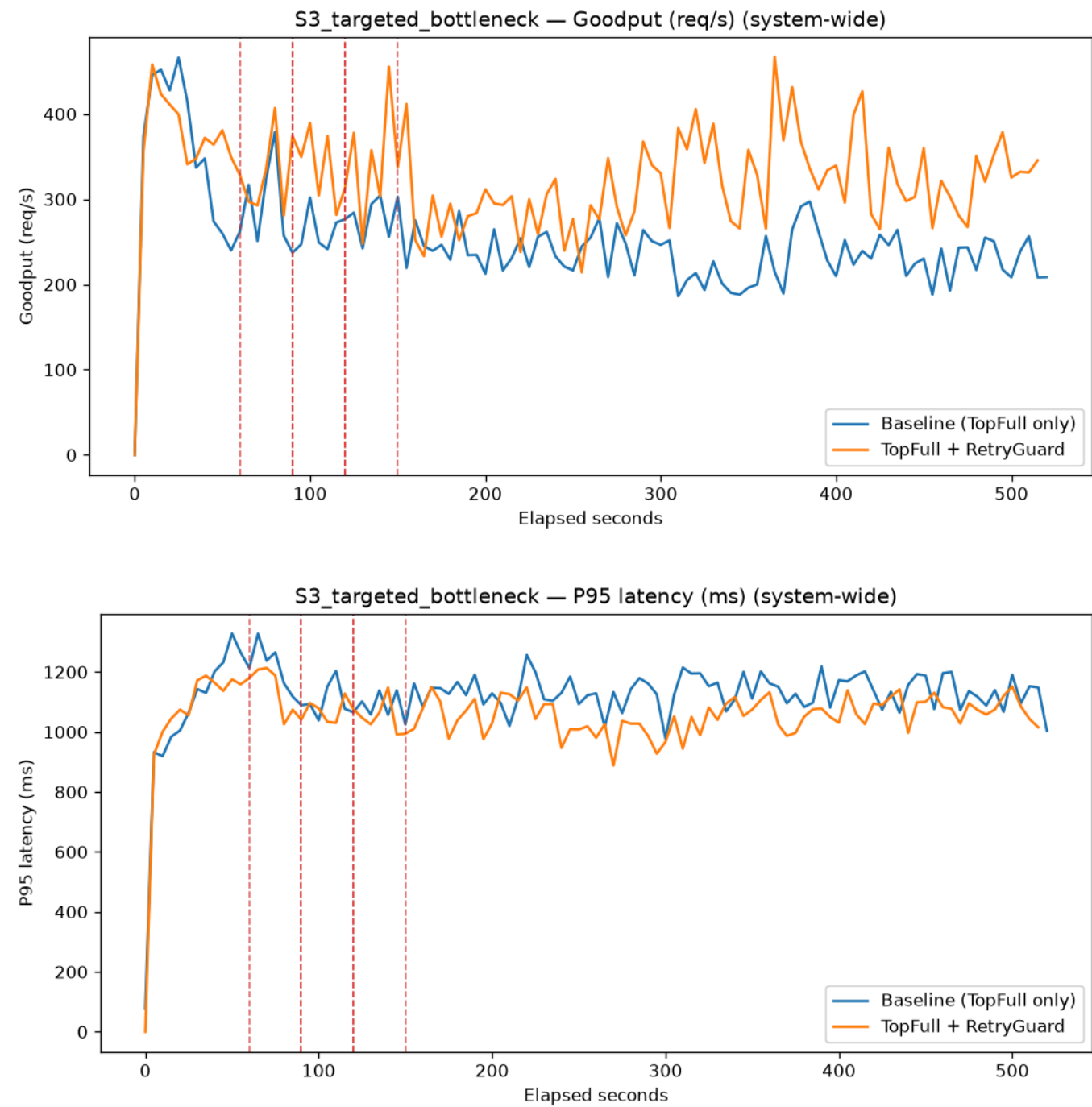
- After the first ~30s, RetryGuard mean goodput sits above baseline for most of the hold (often ~250–480 vs ~200–350 req/s) and the means converge again after ~450s. Rejection is lower on RetryGuard early (~0.2–0.45 vs ~0.4–0.6) then both remain high (~0.35–0.6) through the rest of the 10-minute hold.
- Three red ON→OFF markers sit at ~60s, ~90s, and ~120s. On the frontend retries chart (~680s window) only `checkoutservice` shows retries — two brief spikes of ~0.12 retries/request at ~135s and ~413s — while `cartservice` and `productcatalogservice` stay at 0. The CPU overlay (RetryGuard run, ~650s) stays near zero until ~50s, then plateaus (frontend ~710–740 millicores) with no lasting step-down; `checkoutservice` shows only brief dips near those retry spikes. Memory per service is mostly flat after a ~50s jump on `frontend` / `checkoutservice`.

- No green OFF→ON appears. System-wide rejection stays elevated in both conditions for the whole flat hold, matching the campaign finding of disables without re-enable.

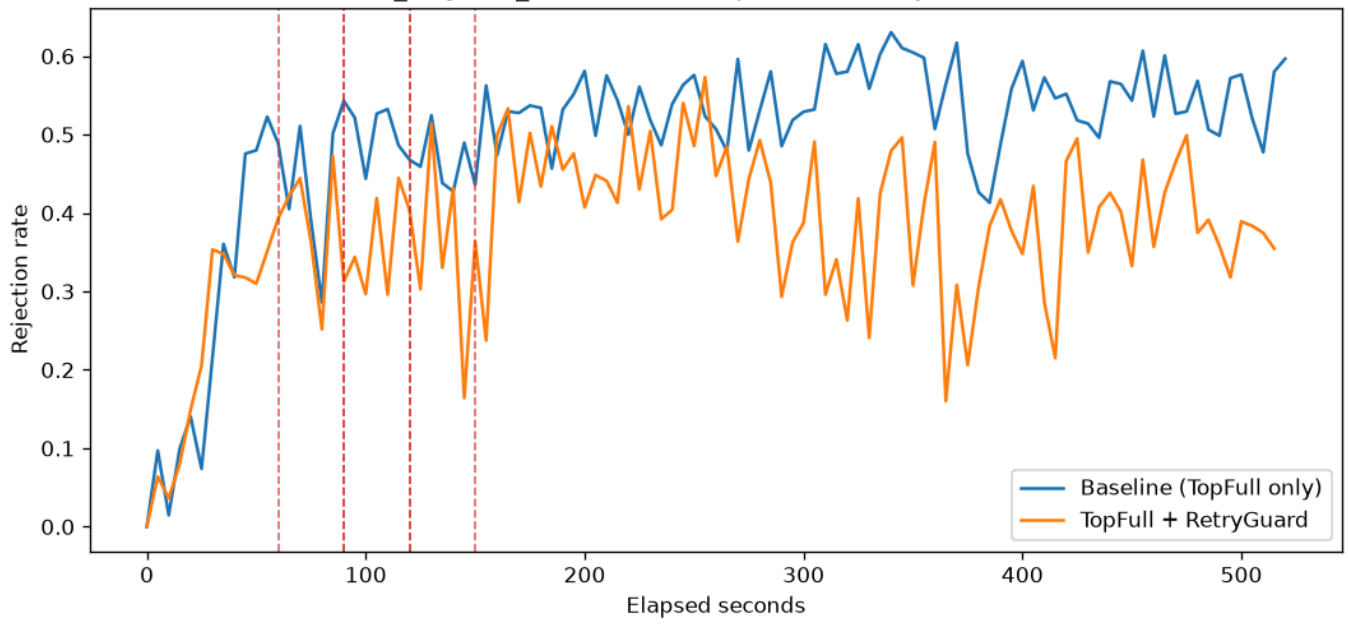
Additional endpoint-level and per-target charts: `charts_gallery/S2_sustained_overload/`.

4.3 Scenario 3 — Targeted Bottleneck (`checkoutservice`)

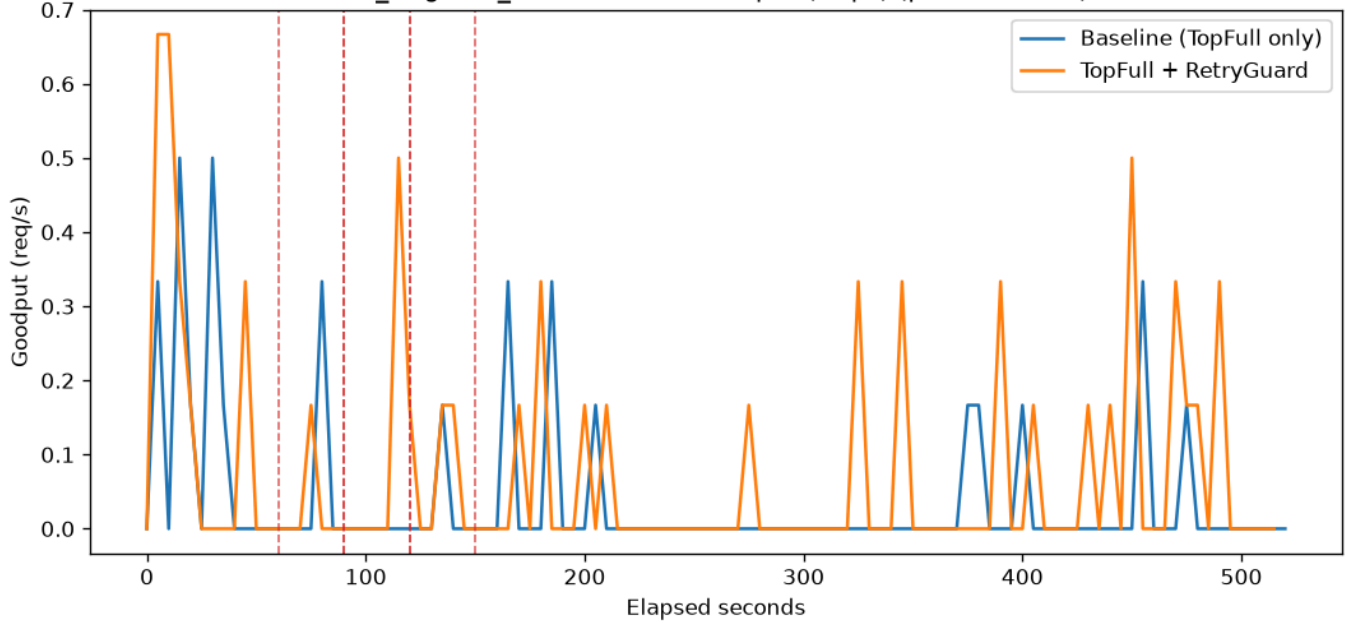
Only `checkoutservice` is CPU-limited; the system-wide view and the checkout-specific view are shown separately since only checkout-routed traffic is directly affected.

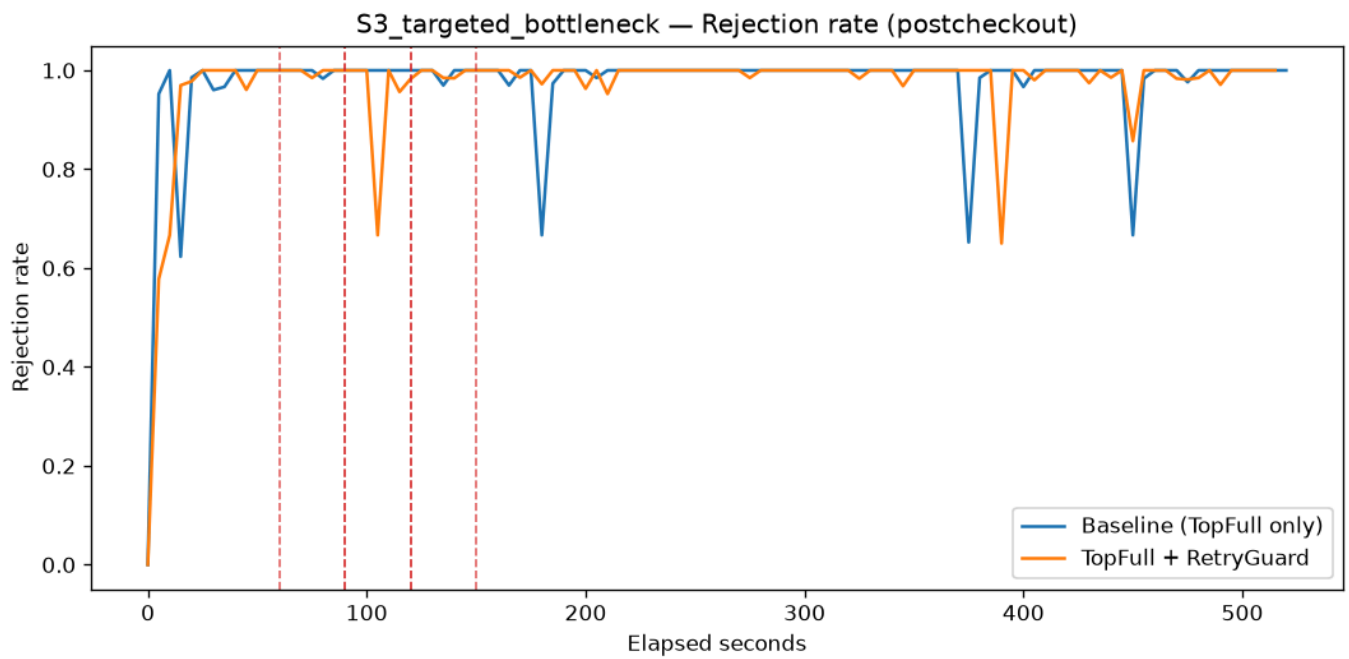


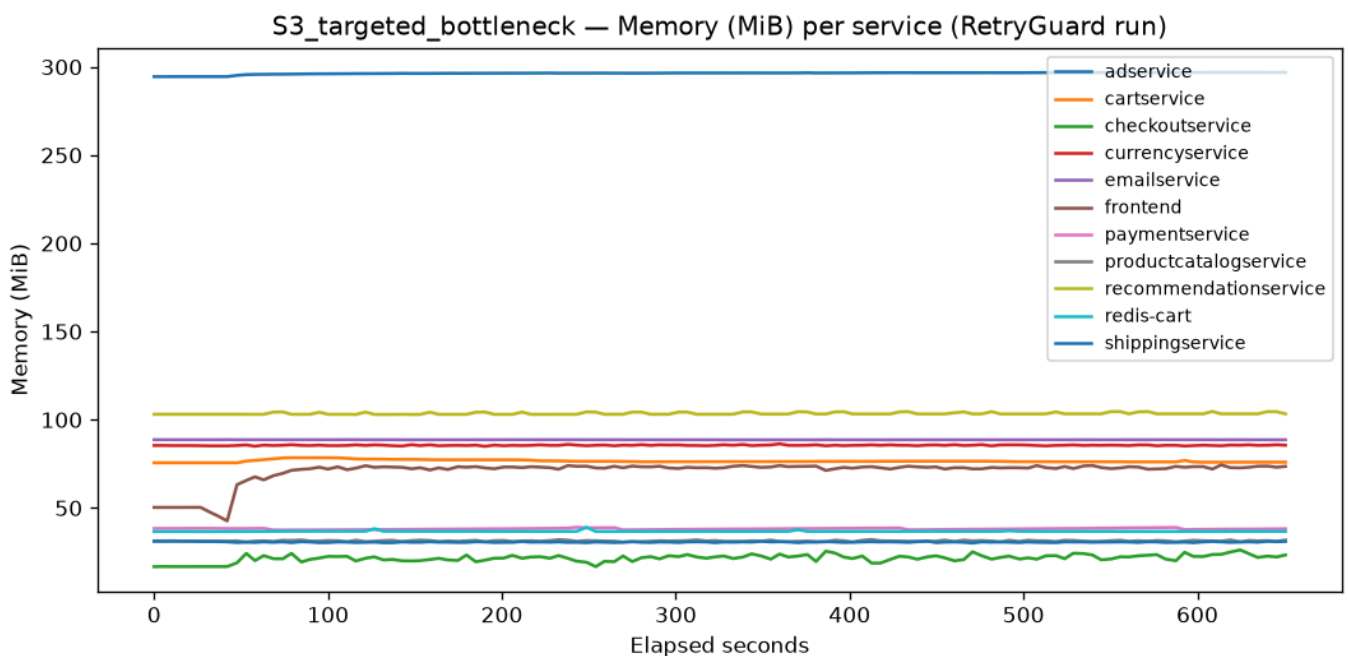
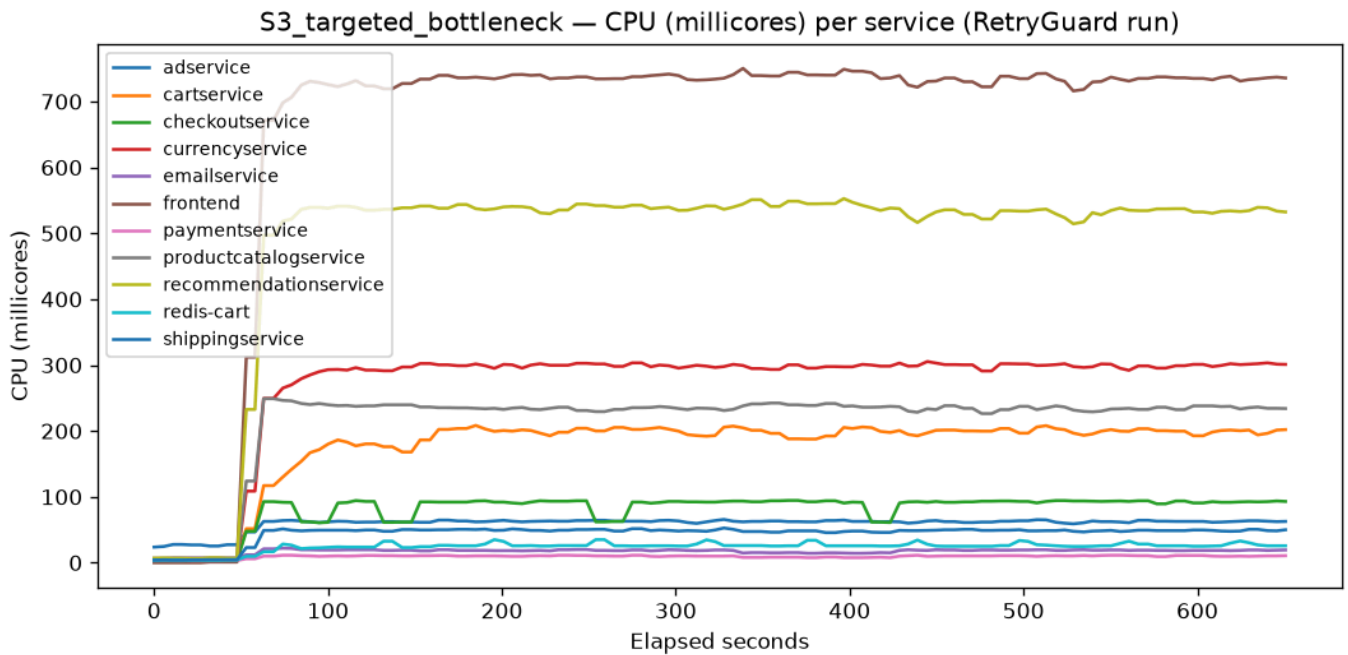
S3_targeted_bottleneck — Rejection rate (system-wide)



S3_targeted_bottleneck — Goodput (req/s) (postcheckout)







What we observe:

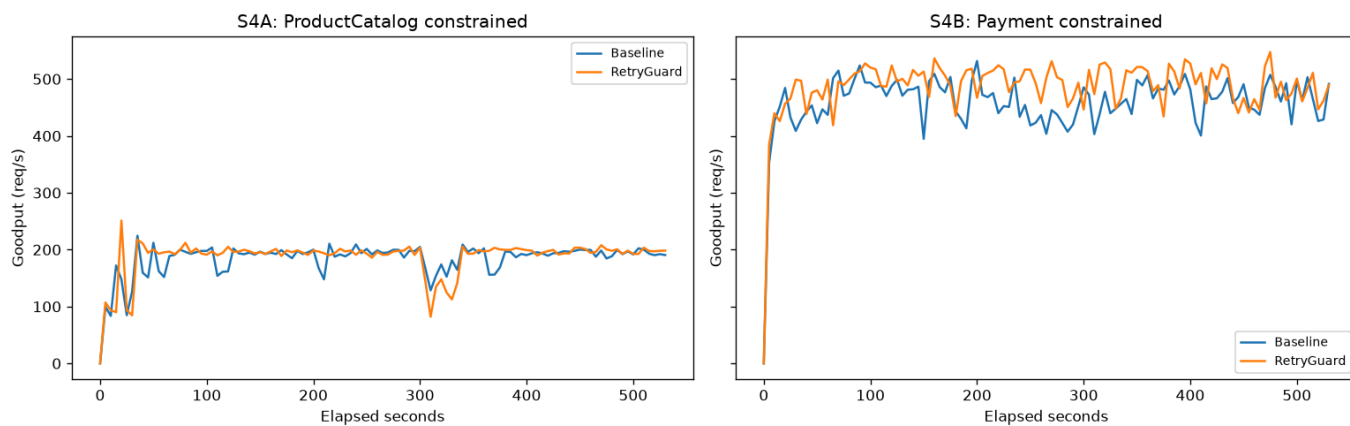
- The checkout-endpoint (`postcheckout`) goodput collapses to near zero for both conditions after an initial ~5s spike (later bursts still <1 req/s), and postcheckout rejection sits near 1.0 in both. The conditions separate on the system-wide chart, not the checkout-endpoint chart: after the ~60s `ON→OFF`, RetryGuard mean goodput stays above baseline (often ~250–450 vs ~180–300 req/s).
- `checkoutservice` CPU on the RetryGuard overlay (~650s) holds at an ~90 millicore plateau (the 100m limit) after a ~50s ramp, with brief dips, not a sustained drop. Checkout-as-caller retries/request to `cartservice`, `paymentservice`, and `productcatalogservice` are flat at 0 for the full ~680s window.
- Frontend retries/request are zero for `cartservice` and `productcatalogservice`. Only the `checkoutservice` target spikes (four peaks ~0.12–0.22 at ~113s, ~157s, ~282s, and ~353s) and then stays at 0 after ~353s.

Additional endpoint-level charts: [charts_gallery/S3_targeted_bottleneck/](#).

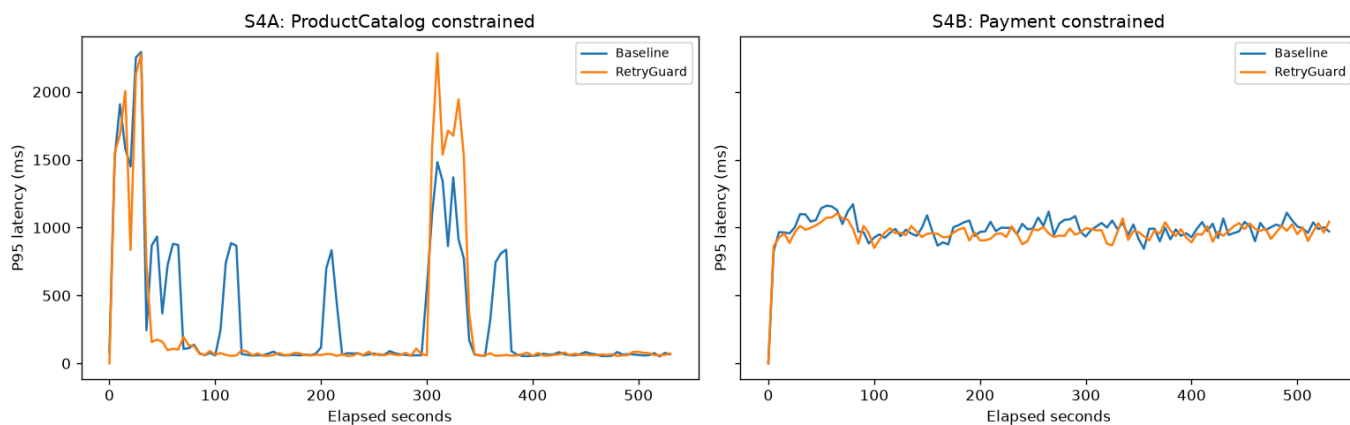
4.4 Scenario 4 — Topology Position (A: ProductCatalog vs B: Payment)

Same constraint method, different position in the call graph — A is gateway-adjacent (Frontend calls it directly), B is Checkout-mediated (Frontend → Checkout → Payment).

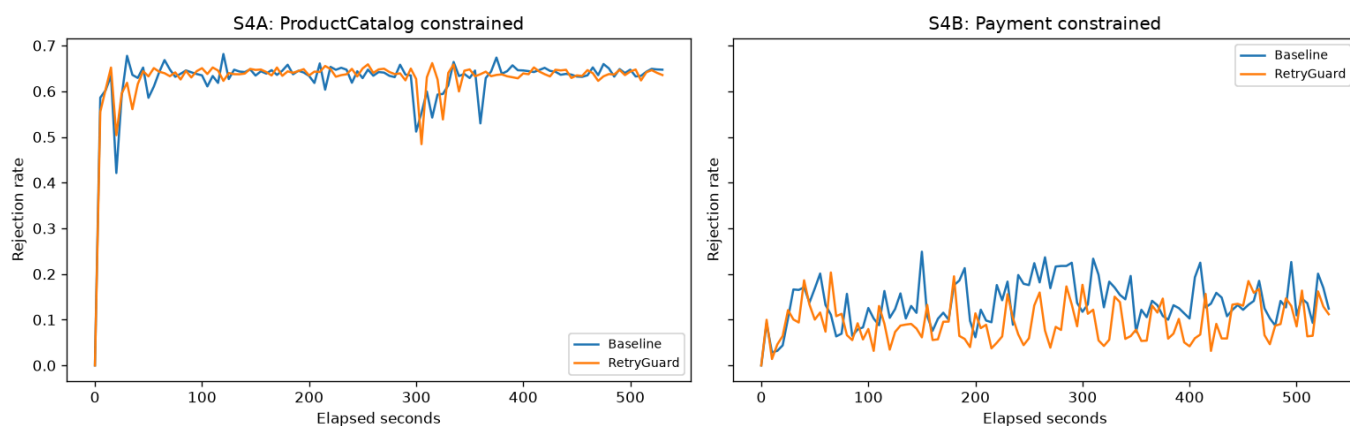
Scenario 4 — Goodput (req/s) (system-wide)



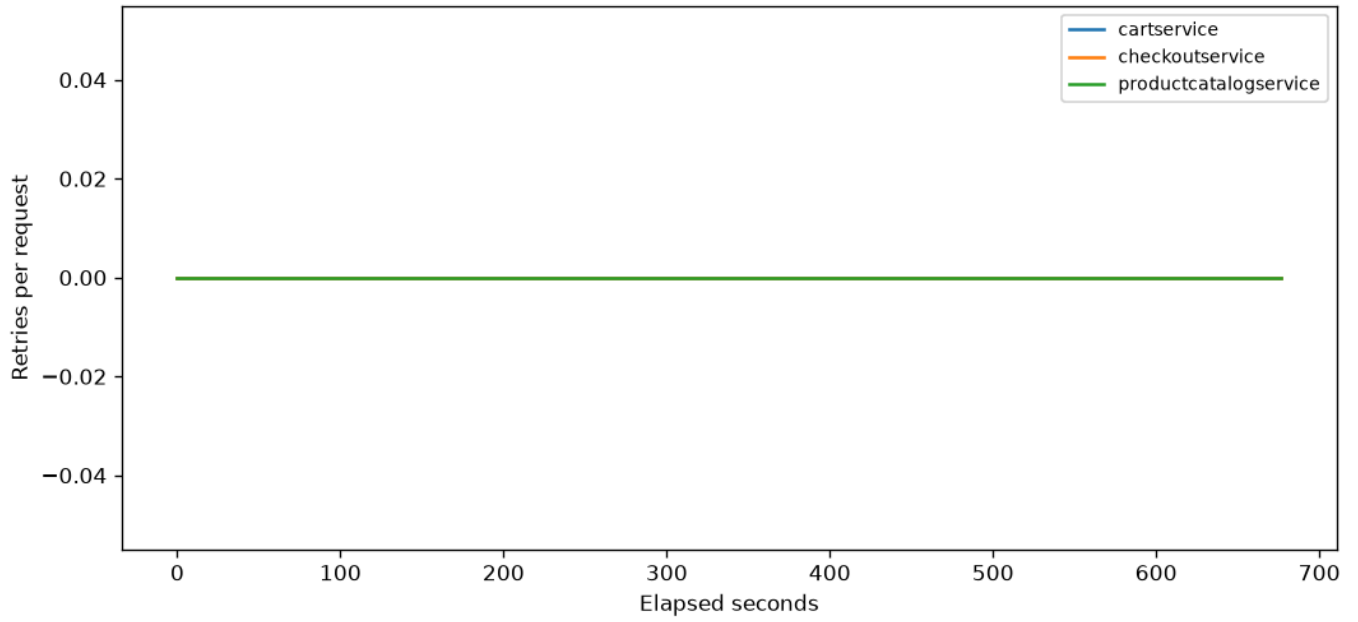
Scenario 4 — P95 latency (ms) (system-wide)



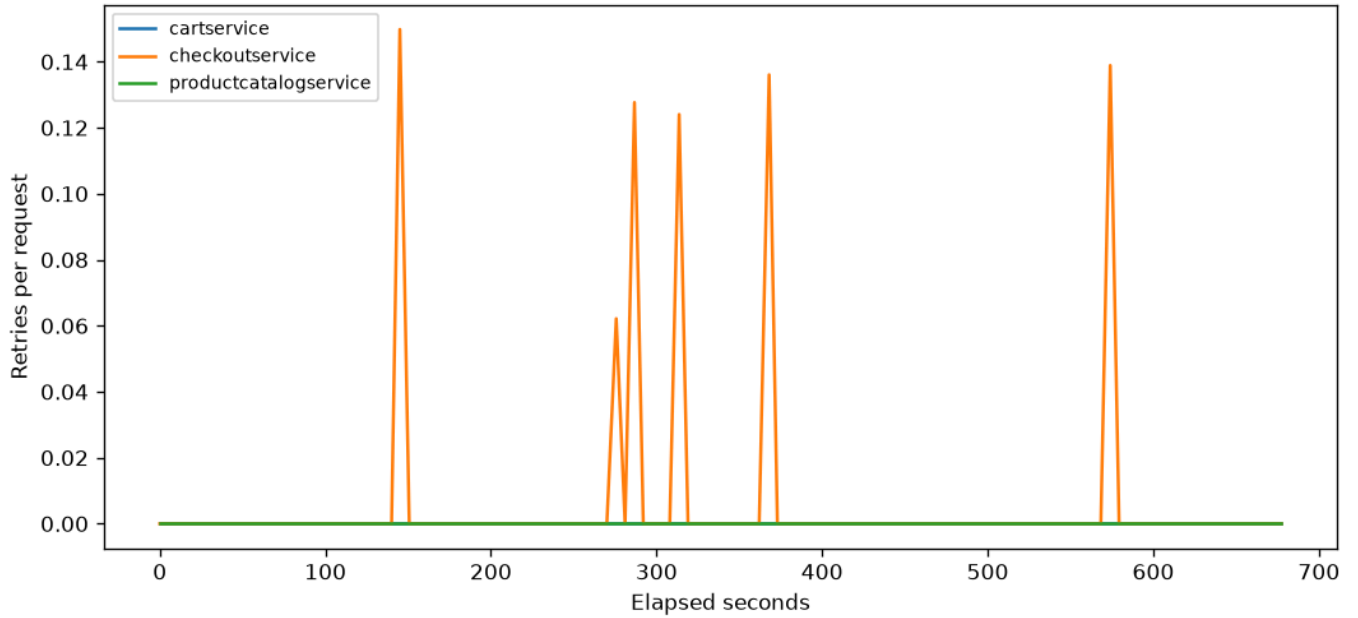
Scenario 4 — Rejection rate (system-wide)



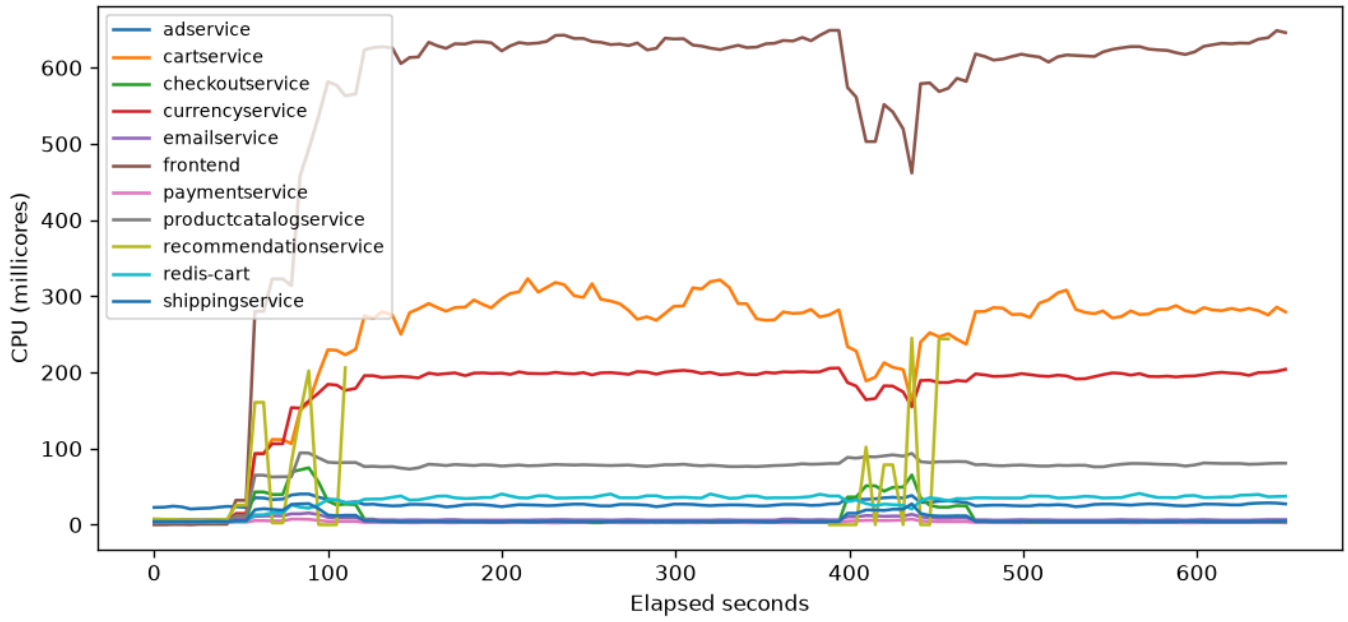
S4A_topology_position_A — envoy_retries_frontend retries/request per target



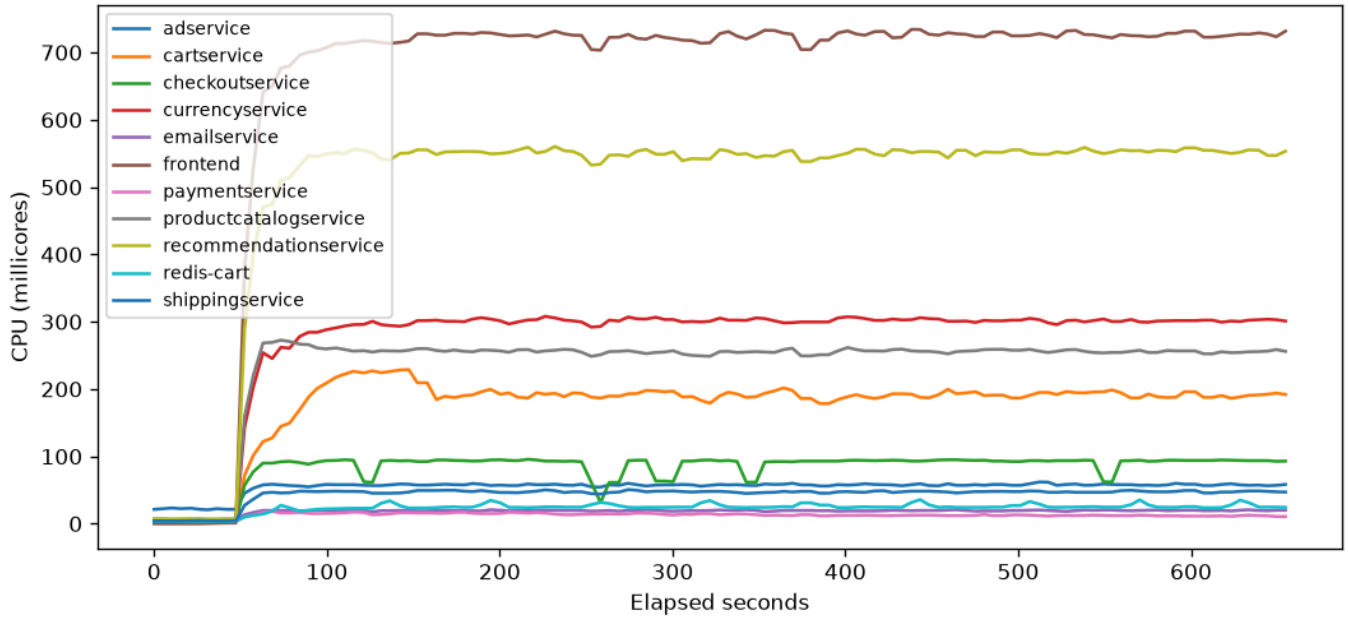
S4B_topology_position_B — envoy_retries_frontend retries/request per target



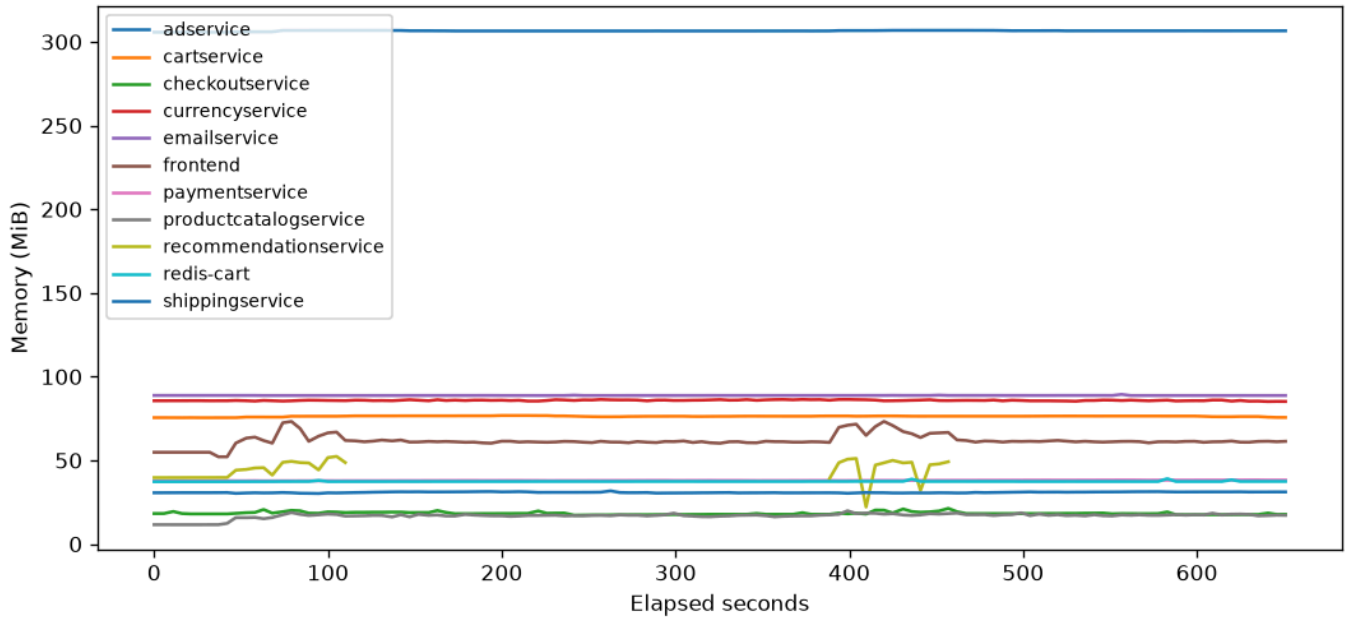
S4A_topology_position_A — CPU (millicores) per service (RetryGuard run)



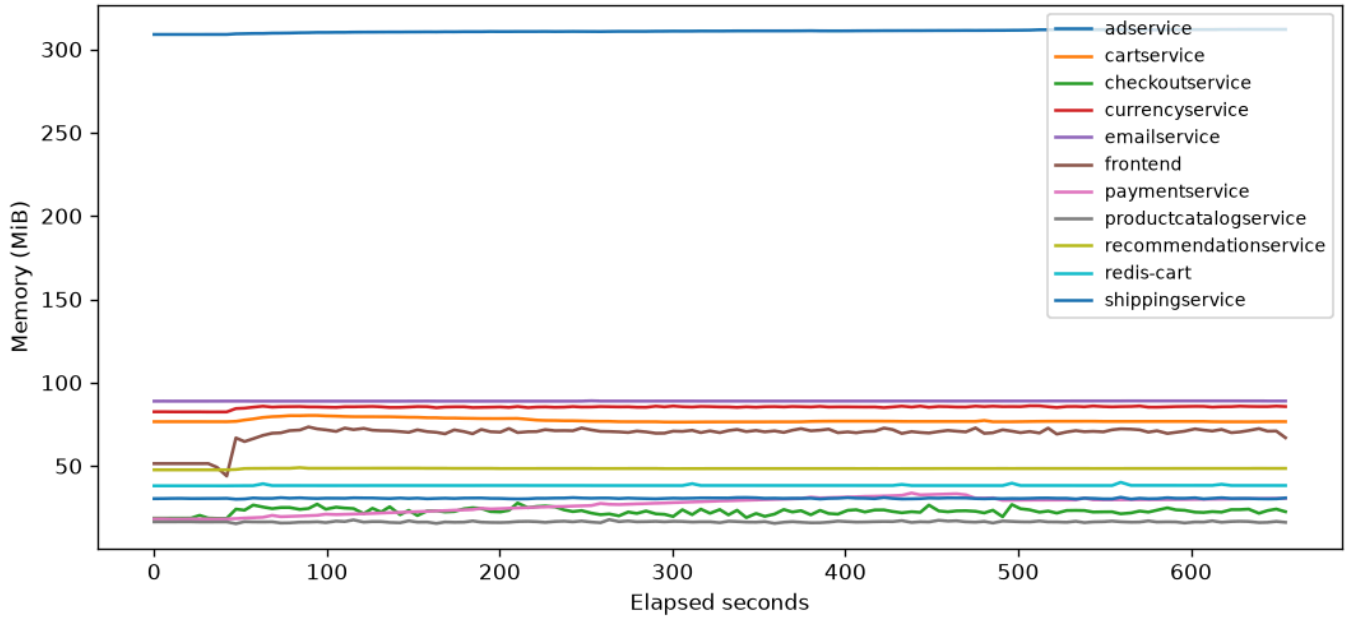
S4B_topology_position_B — CPU (millicores) per service (RetryGuard run)



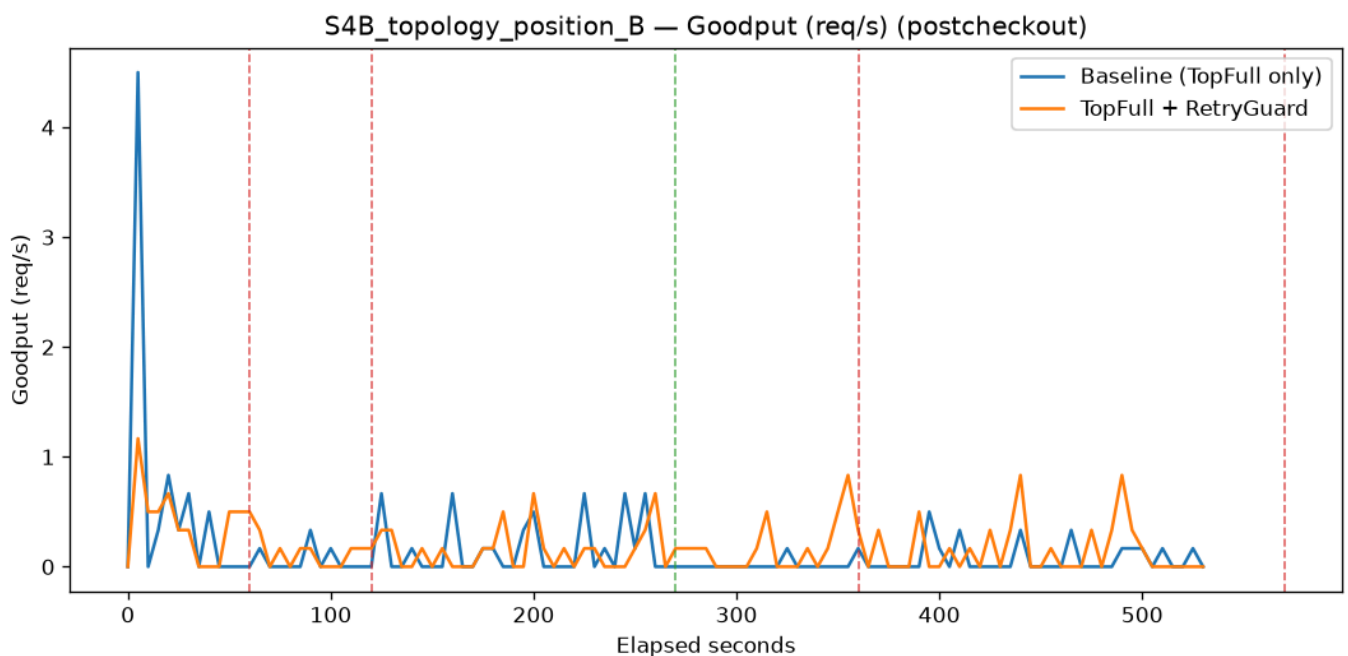
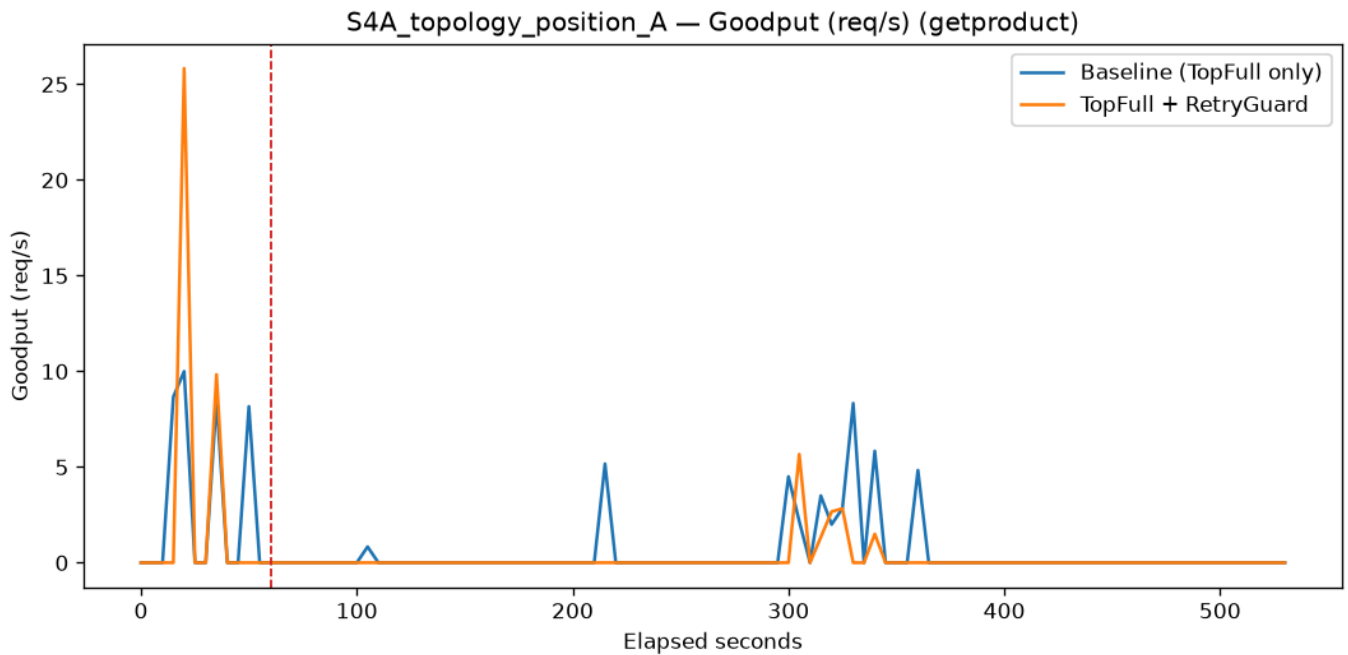
S4A_topology_position_A — Memory (MiB) per service (RetryGuard run)



S4B_topology_position_B — Memory (MiB) per service (RetryGuard run)



Per-position bottleneck-endpoint detail:



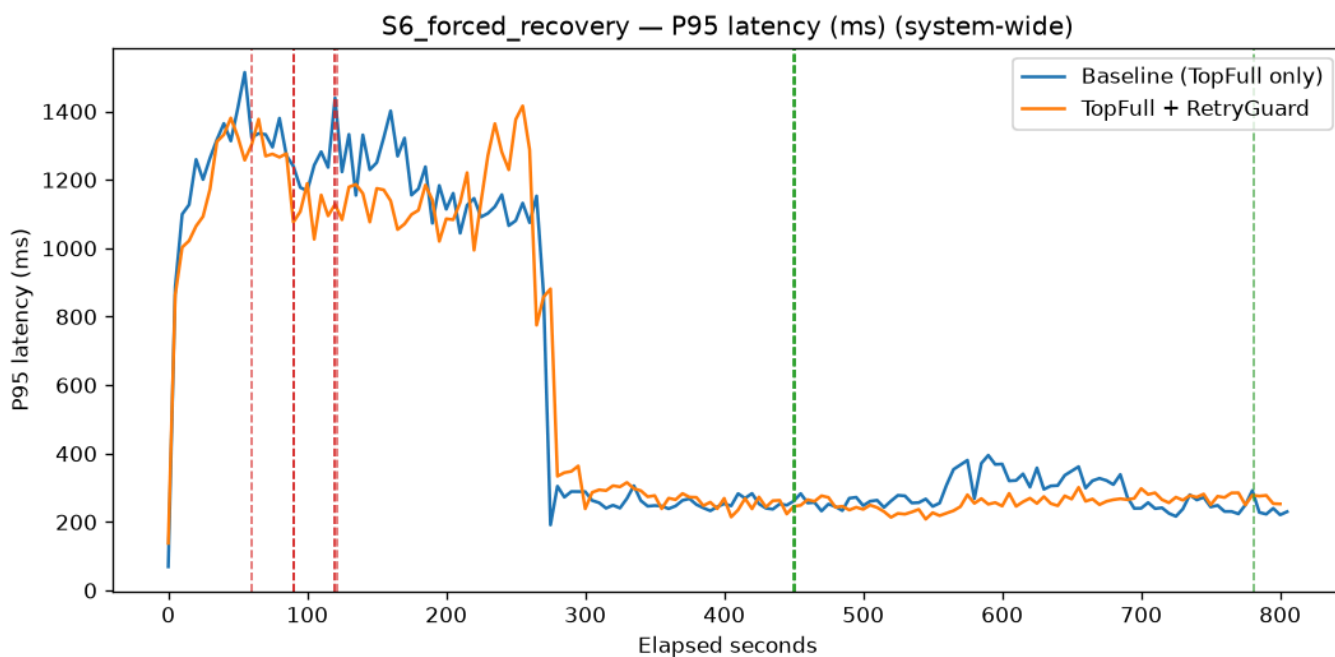
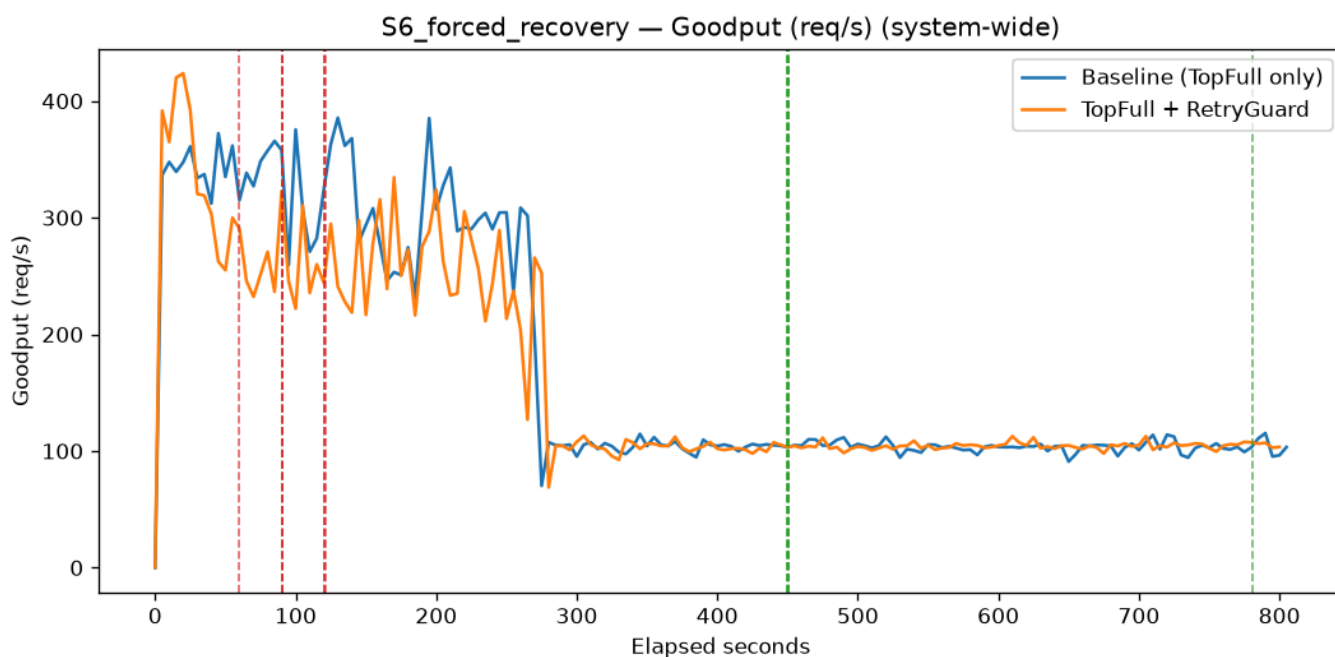
What we observe:

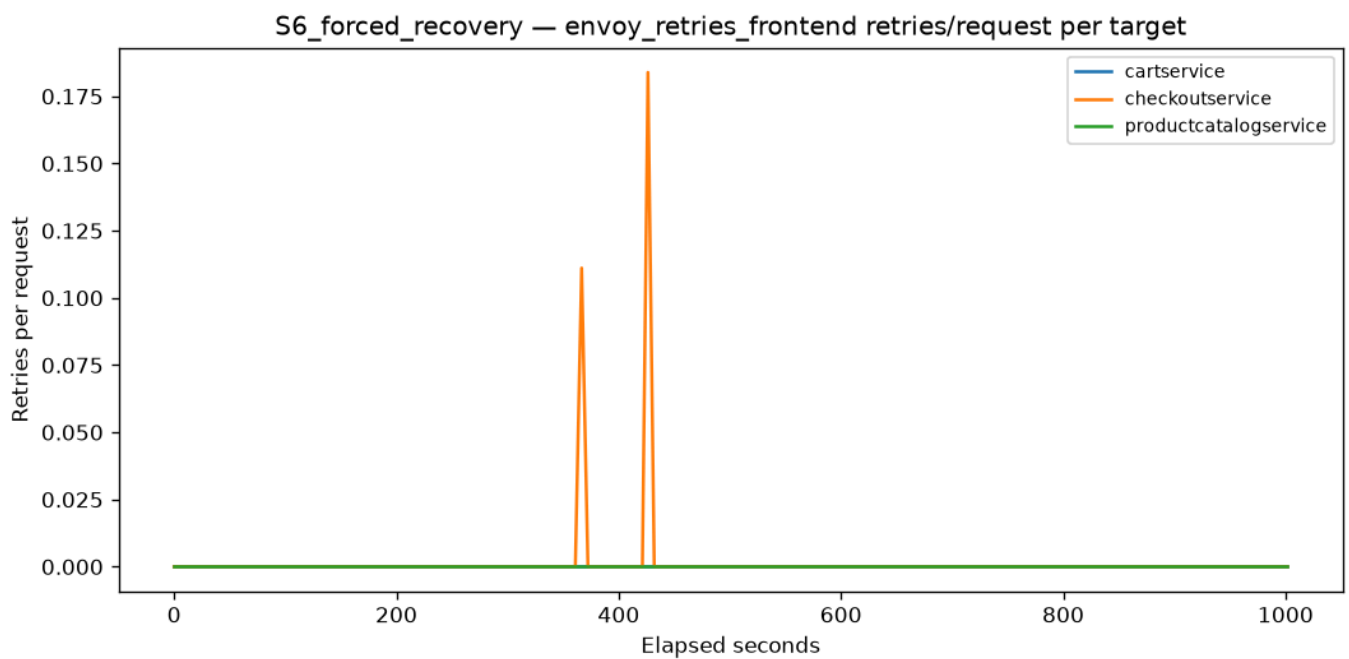
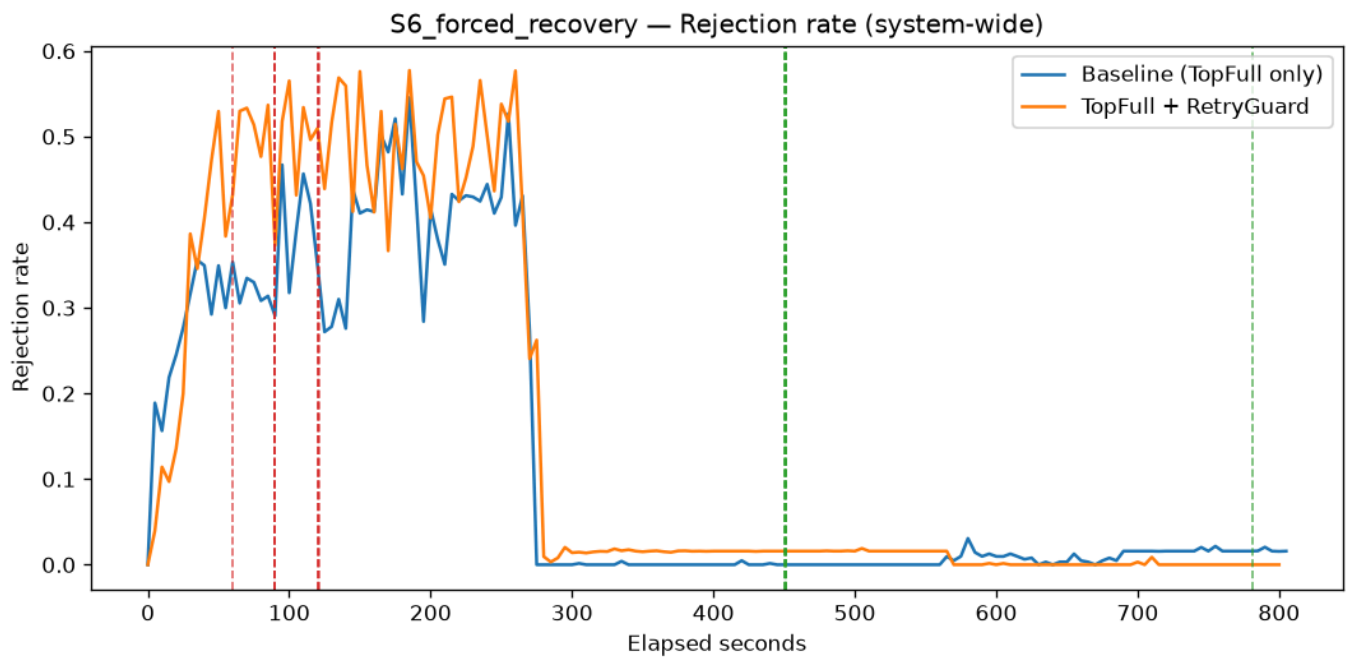
- On system-wide goodput and rejection, the baseline–RetryGuard gap is larger in B than in A. A both hold ~ 200 req/s / ~ 0.65 rejection (RetryGuard smoother; baseline has periodic dips to ~ 150 req/s). B has RetryGuard holding ~ 480 – 530 req/s with rejection ~ 0.05 – 0.15 , while baseline sags after ~ 200 s and rejection ~ 0.1 – 0.25 .
- S4A charts show a red ON→OFF at ~ 60 s; every S4A RetryGuard repeat disables `cartservice`, `checkoutservice`, and `productcatalogservice` together. S4B charts also show a red ON→OFF at ~ 60 s; run4 and run5 disable `checkoutservice` only, while run6 also disables `cartservice` and `productcatalogservice`.
- The side-by-side S4 PNG has no toggle overlays (that plot does not draw them). S4A endpoint charts have no green OFF→ON — S4A logs have no re-enable on any repeat. S4B endpoint charts show a green `cartservice` OFF→ON from run6 (then `cartservice` ON→OFF again) under this flat hold.

Additional endpoint-level charts: [charts_gallery/S4A_topology_position_A/](#) and [charts_gallery/S4B_topology_position_B/](#).

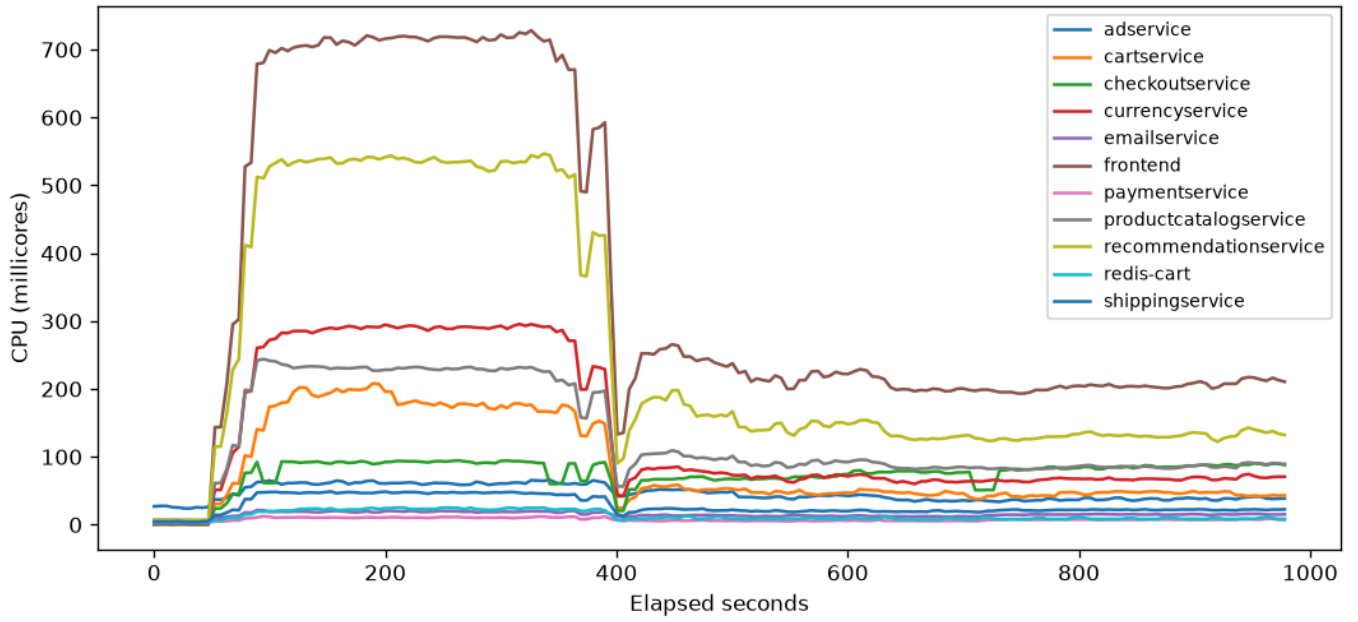
4.5 Scenario 6 — Forced Recovery (+ Scenario 5 Interval Sensitivity)

Peak load for 5 minutes (enough to trigger disable), then dropped to ~25% for 10 minutes so rejection can fall and RetryGuard can re-enable. Scenario 5 reruns this same load shape while sweeping RetryGuard's re-enable window (10/20/30/60s) — it has no baseline of its own, so it's shown here against S6.

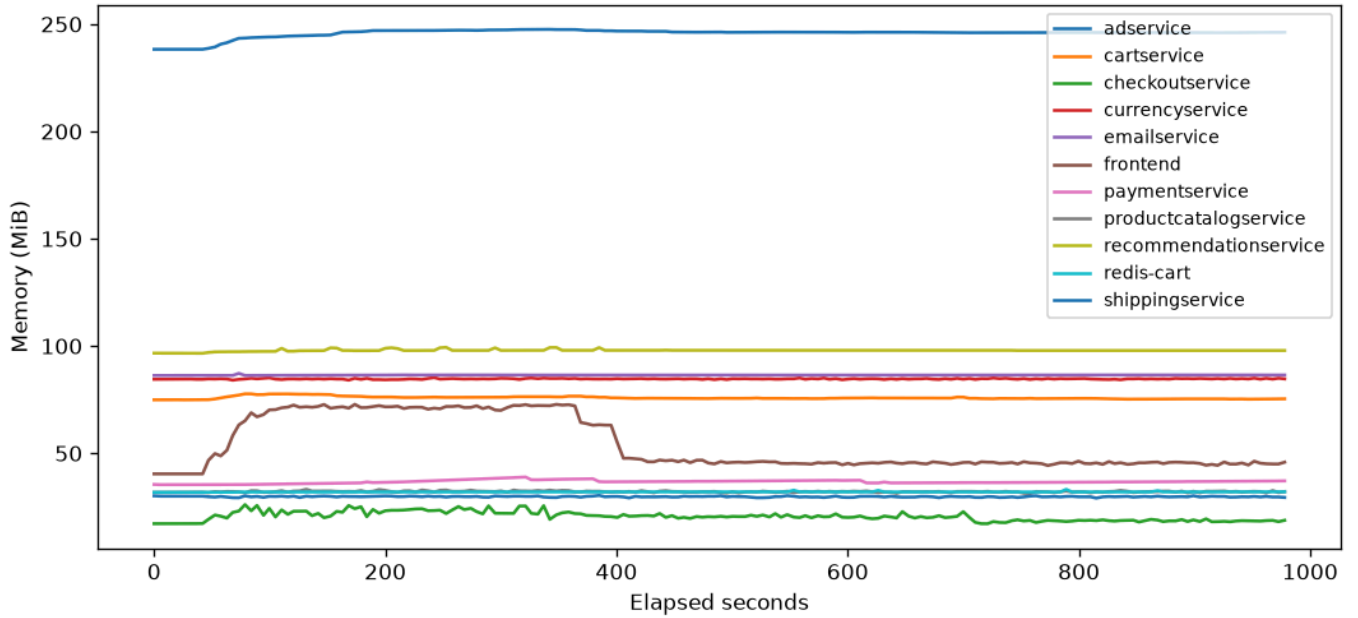




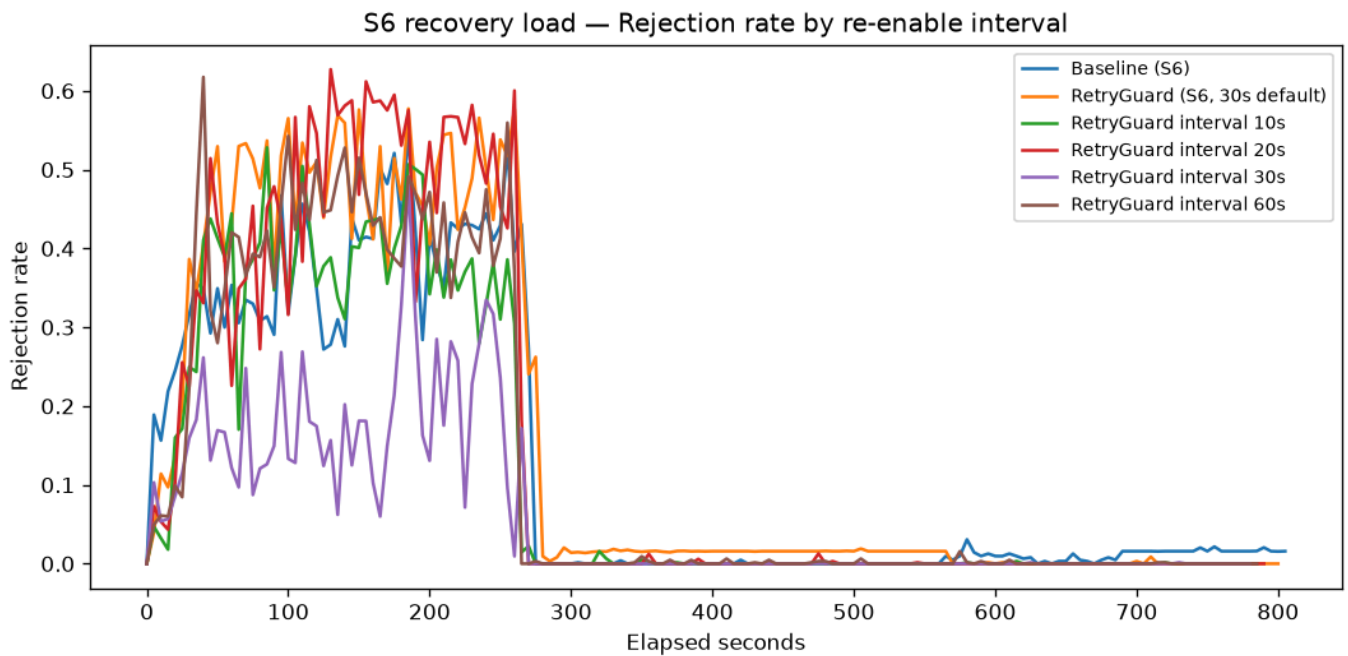
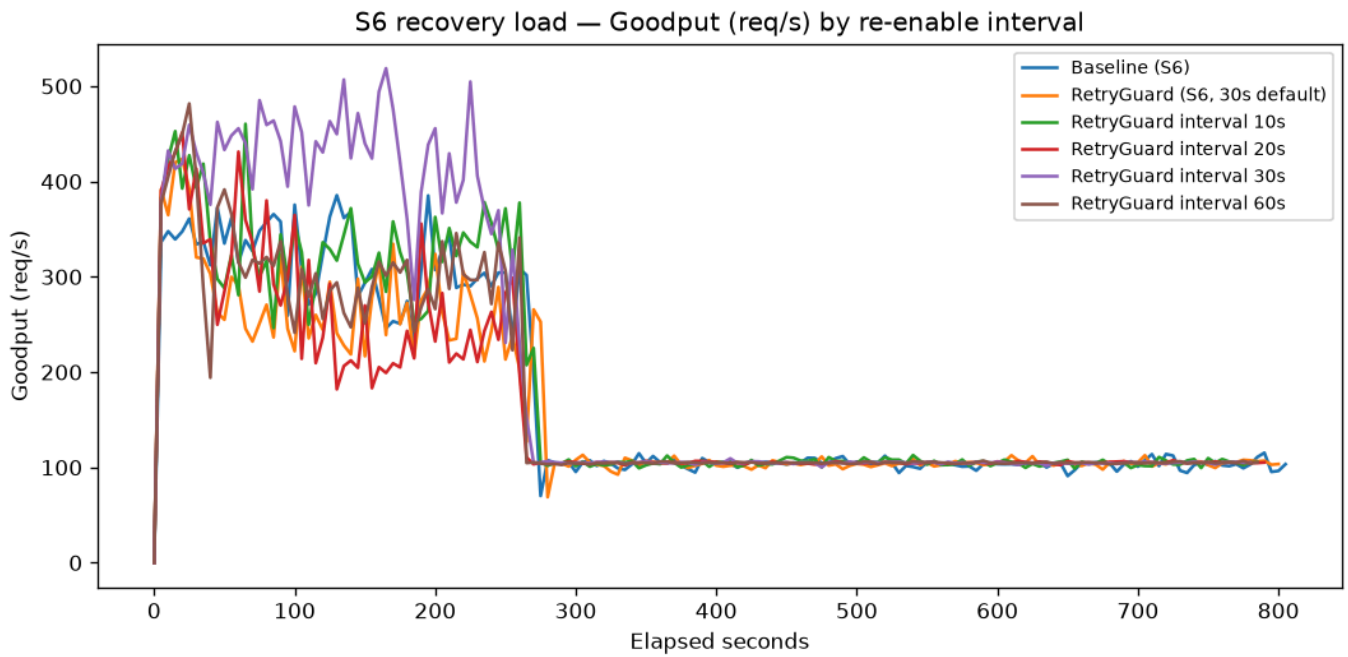
S6_forced_recovery — CPU (millicores) per service (RetryGuard run)



S6_forced_recovery — Memory (MiB) per service (RetryGuard run)



Interval sensitivity (Scenario 5, same load as S6):



Toggle-event timeline (disable/re-enable timestamps per run group):

Run group	Toggle events (elapsed s, service, direction)
Baseline (S6)	n/a
RetryGuard (S6, 30s default)	run_topfull_retryguard_forced_recovery_run1: 60s checkoutservice ON→OFF; 90s cartservice ON→OFF; 90s productcatalogservice ON→OFF; 450s cartservice OFF→ON; 450s productcatalogservice OFF→ON; 781s checkoutservice OFF→ON run_topfull_retryguard_forced_recovery_run2: 60s checkoutservice ON→OFF; 120s cartservice ON→OFF; 120s productcatalogservice ON→OFF; 451s cartservice OFF→ON; 451s checkoutservice OFF→ON; 451s productcatalogservice OFF→ON run_topfull_retryguard_forced_recovery_run3: 90s checkoutservice ON→OFF; 120s cartservice ON→OFF; 121s productcatalogservice ON→OFF; 451s cartservice OFF→ON; 451s checkoutservice OFF→ON; 451s productcatalogservice OFF→ON

Run group	Toggle events (elapsed s, service, direction)
RetryGuard interval 10s	run_topfull_retryguard_interval_10s_run3: 60s checkoutservice ON→OFF; 120s cartservice ON→OFF; 150s productcatalogservice ON→OFF; 181s cartservice OFF→ON; 181s productcatalogservice OFF→ON; 271s cartservice ON→OFF; 271s productcatalogservice ON→OFF; 301s cartservice OFF→ON; 301s productcatalogservice OFF→ON; 391s checkoutservice OFF→ON run_topfull_retryguard_interval_10s_run4: 60s checkoutservice ON→OFF; 90s cartservice ON→OFF; 90s productcatalogservice ON→OFF; 391s cartservice OFF→ON; 391s checkoutservice OFF→ON; 391s productcatalogservice OFF→ON run_topfull_retryguard_interval_10s_run5: 60s checkoutservice ON→OFF; 90s cartservice ON→OFF; 120s productcatalogservice ON→OFF; 390s cartservice OFF→ON; 390s checkoutservice OFF→ON; 390s productcatalogservice OFF→ON
RetryGuard interval 20s	run_topfull_retryguard_interval_20s_run3: 60s checkoutservice ON→OFF; 90s cartservice ON→OFF; 120s productcatalogservice ON→OFF; 421s cartservice OFF→ON; 421s checkoutservice OFF→ON; 421s productcatalogservice OFF→ON run_topfull_retryguard_interval_20s_run4: 60s checkoutservice ON→OFF; 90s cartservice ON→OFF; 120s productcatalogservice ON→OFF; 420s cartservice OFF→ON; 420s checkoutservice OFF→ON; 420s productcatalogservice OFF→ON run_topfull_retryguard_interval_20s_run5: 60s checkoutservice ON→OFF; 90s cartservice ON→OFF; 90s productcatalogservice ON→OFF; 420s cartservice OFF→ON; 420s checkoutservice OFF→ON; 420s productcatalogservice OFF→ON
RetryGuard interval 30s	run_topfull_retryguard_interval_30s_run3: 60s checkoutservice ON→OFF; 121s cartservice ON→OFF; 211s cartservice OFF→ON; 271s cartservice ON→OFF; 421s cartservice OFF→ON; 451s checkoutservice OFF→ON run_topfull_retryguard_interval_30s_run4: 60s checkoutservice ON→OFF; 120s cartservice ON→OFF; 210s productcatalogservice ON→OFF; 450s cartservice OFF→ON; 450s checkoutservice OFF→ON; 450s productcatalogservice OFF→ON run_topfull_retryguard_interval_30s_run5: 60s checkoutservice ON→OFF; 120s cartservice ON→OFF; 120s productcatalogservice ON→OFF; 420s cartservice OFF→ON; 420s productcatalogservice OFF→ON; 450s checkoutservice OFF→ON
RetryGuard interval 60s	run_topfull_retryguard_interval_60s_run3: 60s checkoutservice ON→OFF; 120s cartservice ON→OFF; 121s productcatalogservice ON→OFF; 541s cartservice OFF→ON; 541s checkoutservice OFF→ON; 541s productcatalogservice OFF→ON run_topfull_retryguard_interval_60s_run4: 60s checkoutservice ON→OFF; 120s cartservice ON→OFF; 180s productcatalogservice ON→OFF; 450s productcatalogservice OFF→ON; 510s cartservice OFF→ON; 540s checkoutservice OFF→ON run_topfull_retryguard_interval_60s_run5: 60s checkoutservice ON→OFF; 90s cartservice ON→OFF; 120s productcatalogservice ON→OFF; 540s cartservice OFF→ON; 540s checkoutservice OFF→ON; 540s productcatalogservice OFF→ON

What we observe:

- After the load drop at ~270–280s, RetryGuard goodput falls with baseline to a flat ~100 req/s and the two lines overlay for the remaining ~10 minutes — recovery is simultaneous at the same level, not faster or slower on RetryGuard. During the peak window, baseline mean goodput is above RetryGuard. Green OFF→ON markers at ~450s and ~780s do not change the goodput level; red ON→OFF markers at ~60s and ~90s sit in the high-load window.
- Extra disable/re-enable pairs appear on the 10s interval run3 (5 ON→OFF and 5 OFF→ON, including a second cartservice / productcatalogservice disable–re-enable cycle before checkout re-enables at 391s). Other 10/20/60s repeats and S6 RetryGuard are one disable then re-enable per service; 30s run3 also toggles cartservice twice (3 ON→OFF / 3 OFF→ON total).
- After the drop, all six series sit on top of each other at ~100 req/s goodput and ~0 rejection; the 30s paper-default is not separated from 10s/20s/60s in that recovery window. During the peak window the S5 30s mean sits higher than the other interval lines; S6's own 30s RetryGuard line does not.

Additional endpoint-level charts: [charts_gallery/S6_forced_recovery/](#) (includes S5 interval-sensitivity overlays).